Đề 06 · Nhập môn Công nghệ nền tảng (RIPT1304) · Tài liệu học nội bộ, không nộp

Hiểu và làm chủ phần việc của Bùi Thị Vân Anh

Sinh viên 3 · QA Lead, Performance Test Engineer, phân tích số liệu

Mã sinh viên B24DCCC007 · Lớp RIPT1304-20261-01

Bạn giữ phần đo: viết kịch bản giả lập sinh viên, chạy 3 mức tải, đọc kết quả, tổng hợp thành bảng và chỉ ra điểm nghẽn. Mọi con số trong báo cáo nhóm đều đi qua tay bạn, nên bạn phải giải thích được mỗi số đo ra sao và nói lên điều gì.

Mục lục

Phần 0. Bắt đầu từ con số 0: dòng lệnh, Git, Docker

- 0.1 Cách đọc tài liệu này
- 0.2 Lab là gì, vì sao làm hỏng cũng không sao
- 0.3 Terminal và các lệnh tối thiểu
- 0.4 Cài công cụ (làm 1 lần)
- 0.5 Git: lấy mã về, xem mình đã sửa gì, hoàn tác
- 0.6 Mở và sửa tệp bằng VS Code
- 0.7 Docker: lab nằm ở đâu trên máy bạn

Phần 1. Kiến thức nền cả nhóm phải nắm

- 1.1 Đề bài yêu cầu gì
- 1.2 Hệ thống gồm những phần nào (ví như một quán ăn)
- 1.3 Từ điển thuật ngữ (đọc kỹ, gặp lại ở khắp tài liệu)
- 1.4 Lab của nhóm trông như thế nào
- 1.5 Repo này có gì, phần của mình ở đâu
- 1.6 Dựng lab trên máy của bạn
- 1.7 Quy tắc chung khi sửa cấu hình
- 1.8 Lỗi hay gặp khi dựng lab và cách xử lý
- 1.9 Kubernetes cho người chưa dùng bao giờ
- 1.10 Đọc một tệp YAML của Kubernetes
- 1.11 Đọc một script bash
- 1.12 Bản đồ thư mục infra: tệp nào để làm gì
- 1.13 Xem cụm đang chạy: các lệnh kubectl cơ bản
- 1.14 Buổi thực hành đầu tiên (khoảng 1 giờ)
- 1.15 Đọc Grafana, Prometheus, Alertmanager

Phần 2. Phần việc của bạn

- 2.1 Phiếu đề giao gì cho bạn
- 2.2 Sản phẩm của bạn trong hồ sơ

Phần 3. Kiểm thử hiệu năng và k6 từ đầu

- 3.1 Kiểm thử hiệu năng là gì, khác kiểm thử chức năng thế nào
- 3.2 k6 hoạt động thế nào
- 3.3 Các kịch bản của nhóm
- 3.4 Mô hình mở và mô hình đóng (câu hỏi rất hay bị hỏi)
- 3.5 Phân vị p50, p95, p99 và độ lệch
- 3.6 Thông lượng, điểm gãy, điểm nghẽn

Phần 4. Kiểm thử 2: đo, đọc kết quả, tổng hợp

- 4.1 Bước 1: đo thăm dò để chọn 3 mức
- 4.2 Bước 2: chạy Kiểm thử 2 cho đúng cách
- 4.3 Đọc kết quả k6 (tệp thật, mức 36 lượt/giây, lần 1)
- 4.4 Bước 3: tổng hợp
- 4.5 Đo thêm
- 4.6 Vấn đề đã gặp: lỗi của phép đo, không phải lỗi hệ thống
- 4.7 Đọc bảng kiểm thử

Phần 5. Sửa và cấu hình trực tiếp

- 5.1 Đổi mức tải, thời gian đo
- 5.2 Đổi 3 mức của Kiểm thử 2, số lần mỗi mức
- 5.3 Đổi tiêu chí đạt
- 5.4 Cho thầy xem 1 lần đo bất kỳ

Phần 6. Chạy lại và số liệu của bạn
6.1 Chạy lại phần của bạn, từng bước
6.2 Số liệu để nói khi bảo vệ
Phần 7. Câu hỏi vấn đáp
Câu hỏi về phần của bạn
Câu hỏi chung ai cũng phải trả lời được
Phần 8. Việc cần làm trước khi nộp và bảo vệ
Trước khi nộp
Trình bày 3 phút
Trước buổi bảo vệ

> Cách đọc: chưa dùng dòng lệnh, Git, Docker bao giờ thì bắt đầu từ **Phần 0**; sau đó phần 1 (kiến thức chung, Kubernetes, buổi thực hành đầu tiên), rồi lần lượt các phần riêng của bạn. Chỗ nào có **[mở trên GitHub]** là bấm được, mở đúng tệp và đúng dòng trong repo. Đoạn cấu hình trong tài liệu được trích thẳng từ tệp thật, số dòng bên trái là số dòng trong tệp.

Phần 0. Bắt đầu từ con số 0: dòng lệnh, Git, Docker

0.1 Cách đọc tài liệu này

Tài liệu viết cho người **chưa từng** dùng dòng lệnh, Docker hay Kubernetes. Đọc theo thứ tự: **Phần 0** (công cụ, làm 1 lần) → **Phần 1** (kiến thức chung, Kubernetes, buổi thực hành đầu tiên) → các phần riêng của bạn → câu hỏi vấn đáp → trình bày 3 phút. Đừng nhảy cóc: phần sau dùng từ ngữ của phần trước.

Quy ước trong tài liệu:

Bạn thấy	Nghĩa là
Khối nền xám chứa lệnh, ví dụ `make status`	Gõ (hoặc dán) **từng dòng** vào terminal rồi bấm Enter
Chữ sau dấu `#` trên cùng dòng lệnh	Ghi chú giải thích, không cần gõ (gõ vào cũng không sao, máy bỏ qua)
`<tên pod>`, `<tệp>`	Thay bằng giá trị thật, **bỏ cả dấu** `< >`
Khối có số dòng bên trái `12 | ...`	Trích nguyên văn từ tệp thật để **đọc**, không phải lệnh để gõ
[mở trên GitHub]	Bấm được: mở đúng tệp, đúng dòng trong repo
Cột "Thấy gì là đúng"	Kết quả bạn phải thấy. Thấy khác thì xem mục 1.8 (lỗi hay gặp) trước khi hỏi

0.2 Lab là gì, vì sao làm hỏng cũng không sao

Lab là bản thu nhỏ của hệ thống, chạy ngay trên máy bạn bên trong Docker, dùng **dữ liệu giả**: 1.000 tài khoản dạng `loadtest-0001@example.test`, mật khẩu sinh ngẫu nhiên lúc dựng. Lab **không nối gì** với web Hanni thật trên VPS và không có dữ liệu người dùng thật, nên bạn **không thể** làm hỏng web thật khi thực hành.

- Rối hay hỏng: `make down` xoá sạch lab, dựng lại khoảng 5 phút (mục 1.6).
- Lỡ sửa sai cấu hình: `git restore <tệp>` đưa tệp về như lúc mới lấy về (mục 0.5), rồi `make deploy`.
- Các script cố ý gây sự cố (`chaos.sh`, `failover.sh`, `kt3-...`) chỉ tác động lên lab.

3 điều **không làm**: không chạy các script này trên VPS hay máy chủ thật; không đưa mật khẩu, tệp `.env` lên GitHub; không sửa tay tệp trong `ho-so-nop/data/raw/` và `ho-so-nop/evidence/` (thầy dùng để đối chiếu số trong báo cáo).

0.3 Terminal và các lệnh tối thiểu

Terminal (cửa sổ dòng lệnh) là nơi gõ lệnh bằng chữ thay cho bấm chuột. Mở terminal:

Máy	Cách mở
Ubuntu	Ctrl+Alt+T
macOS	Cmd+Space, gõ "Terminal", Enter
Windows	Cài WSL theo mục 0.4, rồi mở ứng dụng **Ubuntu** từ menu Start. Mọi lệnh trong tài liệu gõ ở cửa sổ Ubuntu này, **không** gõ trong PowerShell hay CMD

Dòng chờ lệnh có dạng `ten@may:~/hanni-server/infra$` : phần sau dấu `:` là **thư mục đang đứng** (`~` là thư mục nhà của bạn).

Lệnh	Làm gì
`pwd`	In thư mục đang đứng
`ls` , `ls -la`	Liệt kê tệp trong thư mục (`-la` : cả tệp ẩn, kèm dung lượng, quyền)
`cd infra` , `cd ..` , `cd ~`	Vào thư mục `infra` / lên 1 cấp / về thư mục nhà
`cat <tệp>`	In cả tệp ra màn hình
`less <tệp>`	Xem tệp dài, cuộn bằng mũi tên, bấm `q` để thoát
`head -20 <tệp>` , `tail -20 <tệp>`	20 dòng đầu / 20 dòng cuối
`tail -f <tệp>`	Xem tệp đang được ghi thêm, Ctrl+C để thoát
`grep -n "chữ" <tệp>`	Tìm dòng có "chữ", in kèm số dòng
`clear`	Xoá màn hình cho dễ nhìn

Phím nên nhớ: **Tab** tự điền nốt tên tệp, thư mục (gõ `scr` rồi Tab thành `scripts/`); **mũi tên lên** gọi lại lệnh trước; **Ctrl+C** dừng lệnh đang chạy (luôn an toàn); **Ctrl+Shift+C / Ctrl+Shift+V** chép, dán trong terminal Linux (macOS: Cmd+C / Cmd+V); **Ctrl+Shift+T** mở thêm 1 tab terminal.

Ba điều hay làm người mới bối rối:

1. **Đứng sai thư mục.** `scripts/kt1.sh` là đường dẫn **tính từ thư mục đang đứng**, nên phải `cd hanni-server/infra` trước. Báo `No such file or directory` thì gõ `pwd` xem mình đang ở đâu.
2. **Lệnh im lặng là thành công.** Nhiều lệnh xong việc không in gì. Lỗi thì thường có chữ `error` , `Error` , `failed` , `denied` . Gõ `echo $?` ngay sau lệnh để xem **mã thoát**: `0` là thành công, số khác là lỗi.
3. **Lệnh chạy lâu.** k6 đo 5 phút, `make images` lần đầu vài phút: để yên, đừng đóng cửa sổ. Muốn làm việc khác thì mở tab terminal mới.

Chạy script: gõ tên script rồi các tham số, cách nhau bằng dấu cách, ví dụ `scripts/kt1.sh k6/ket-qua/kt1.txt` (tham số là nơi ghi kết quả).

0.4 Cài công cụ (làm 1 lần)

Cần: **Git** (lấy mã về), **make** (chạy nhóm lệnh viết sẵn), **curl**, **python3**, **Docker** kèm **Docker Compose v2**, và nên có **VS Code** để sửa tệp.

Windows 10/11

1. Mở PowerShell bằng quyền quản trị (chuột phải → Run as administrator), gõ `wsl --install` , khởi động lại máy. Mở **Ubuntu** từ menu Start, đặt tên người dùng và mật khẩu Linux (gõ mật khẩu sẽ không hiện ký tự, bình thường).
2. Cài **Docker Desktop for Windows**. Trong Settings: General → bật "Use the WSL 2 based engine"; Resources → WSL integration → bật cho Ubuntu.
3. Trong cửa sổ Ubuntu: `sudo apt update && sudo apt install -y git make curl python3` .
4. RAM: WSL mặc định chỉ được dùng một nửa RAM của máy. Lab cần khoảng 3–5 GB, nên máy 8 GB rất sát, máy 16 GB thoải mái. Muốn tăng: tạo tệp `.wslconfig` trong thư mục người dùng Windows (`C:\Users\<tên>\`), ghi 2 dòng `[wsl2]` và `memory=6GB` , rồi chạy `wsl --shutdown` trong PowerShell.

5. Luôn lấy mã về **thư mục nhà của Ubuntu** (`cd ~`), không để trong `/mnt/c/...` (ổ C của Windows): chậm hơn nhiều và hay lỗi quyền chạy script.

Ubuntu

1. `sudo apt update && sudo apt install -y git make curl python3`
2. Cài Docker bằng script chính thức: `curl -fsSL https://get.docker.com | sudo sh`, rồi `sudo usermod -aG docker $USER`, **đăng xuất và đăng nhập lại** để dùng Docker không cần `sudo`.

macOS: `xcode-select --install` (có sẵn git, make), cài Docker Desktop for Mac, Settings → Resources → Memory từ 6 GB. Nhóm chưa thử lab trên macOS; gặp lỗi thì nhờ Đăng chạy trên GitHub Actions.

Kiểm tra sau khi cài:

Lệnh	Thấy gì là đúng
`git --version`	`git version 2.` …
`make --version`	`GNU Make 4.` … (macOS hiện 3.81 cũng được)
`docker --version`	`Docker version` …
`docker compose version`	`Docker Compose version v2.` … Nếu báo lỗi mà chỉ có `docker-compose` (có gạch nối) thì là bản cũ, cần cài lại Docker
`docker run --rm hello-world`	Có dòng `Hello from Docker!`
`python3 --version`	`Python 3.` …

0.5 Git: lấy mã về, xem mình đã sửa gì, hoàn tác

Git lưu toàn bộ lịch sử thay đổi của mã. **Repo** là thư mục mã kèm lịch sử. **Nhánh** (branch) là một dòng phát triển riêng: mã lab nằm ở nhánh `feat/scaling` của repo `hanni-server`, web thật dùng nhánh `main` nên lab không ảnh hưởng web thật. **Commit** là 1 lần lưu, có mã như `4ef8a31`.

```
cd ~
git clone -b feat/scaling https://github.com/DANG-PH/hanni-server.git   # lấy mã lab về thư mục ~/hanni-server
cd hanni-server
git branch --show-current     # phải in: feat/scaling
git log --oneline -3          # 3 lần lưu gần nhất
git pull                      # lấy bản mới nhất khi nhóm báo có cập nhật
```

Hoàn tác là thứ giúp bạn không sợ sửa sai. Sau khi sửa tệp:

```
git status                            # liệt kê các tệp bạn đã sửa (chữ đỏ, "modified")
git diff                              # xem đúng dòng đã đổi: dòng bắt đầu "-" là cũ, "+" là mới; bấm q để thoát
git restore infra/k8s/app/api.yaml    # đưa 1 tệp về như lúc lấy về (mất phần đã sửa trong tệp đó)
git restore .                         # đưa MỌI tệp về như cũ
```

Đưa tệp cấu hình về như cũ xong thì chạy lại `make deploy` để cụm cũng về như cũ. Bạn **không cần** commit hay push gì lên repo `hanni-server`. Repo tài liệu cong-nghe-nen-tang chỉ để đọc: `git clone https://github.com/DANG-PH/cong-nghe-nen-tang.git` hoặc trên GitHub bấm **Code** → **Download ZIP**.

0.6 Mở và sửa tệp bằng VS Code

1. Cài **VS Code**. Trên Windows cài thêm tiện ích **WSL** (của Microsoft), rồi trong cửa sổ Ubuntu: `cd ~/hanni-server && code .` để mở cả thư mục.
2. Cài tiện ích **YAML** (của Red Hat): gạch đỏ ngay chỗ thụt lề sai trong tệp `.yaml`.
3. Phím hay dùng:

Phím	Làm gì
Ctrl+P, gõ `api.yaml`	Mở tệp theo tên
Ctrl+G, gõ `143`	Nhảy tới dòng 143 (số dòng trong tài liệu này khớp số dòng trong tệp)
Ctrl+F / Ctrl+Shift+F	Tìm trong tệp / tìm trong cả thư mục
Ctrl+S	Lưu
Ctrl+Z	Hoàn tác khi chưa lưu (đã lưu thì dùng `git restore`)
Ctrl+`	Mở terminal ngay trong VS Code, đứng sẵn ở thư mục đang mở

1. Sửa YAML: **chỉ đổi giá trị sau dấu `:`**, giữ nguyên số dấu cách đầu dòng, không thêm phím Tab. Lưu xong mới chạy lệnh áp dụng.
2. Thanh bên trái **Source Control** (biểu tượng nhánh cây) liệt kê tệp đã sửa; bấm vào thấy trước và sau, giống `git diff`.

0.7 Docker: lab nằm ở đâu trên máy bạn

```
Máy của bạn (Ubuntu, WSL trên Windows, hoặc macOS)
└── Docker
    ├── container hanni-lab-server-1  = node k3s-server (172.25.0.2), RAM tối đa 2,8 GB
    │       └── Kubernetes chạy các pod: API, Postgres, Redis, Nginx, Grafana...
    └── container hanni-lab-agent-1   = node k3s-agent  (172.25.0.3), RAM tối đa 1,8 GB
            └── các pod khác (Kubernetes tự chia)
```

- `docker ps` liệt kê container đang chạy: thấy đúng 2 container trên. Các pod nằm **bên trong** 2 container này nên `docker ps` không thấy pod; xem pod bằng `scripts/kubectl` (mục 1.13).
- `docker stats` (Ctrl+C để thoát): CPU, RAM 2 node đang dùng.
- `make stop`: tắt 2 container, **giữ** dữ liệu, trả RAM cho máy. `make start`: bật lại, chờ 1–2 phút cho pod lên đủ.
- `make down`: **xoá** 2 container và ổ đĩa của chúng, mất dữ liệu lab, phải dựng lại từ `make up`.
- Tắt Docker Desktop là lab tắt theo. Khởi động lại máy xong: mở Docker Desktop (Ubuntu: `sudo systemctl start docker` nếu Docker chưa tự chạy), rồi `make start` trong `hanni-server/infra`.

Phần 1. Kiến thức nền cả nhóm phải nắm

1.1 Đề bài yêu cầu gì

Đề 06 của môn yêu cầu: **thiết kế một nền tảng học trực tuyến cho 1.000 sinh viên**. "Thiết kế" ở đây không phải vẽ giao diện, mà là trả lời bằng số các câu hỏi của người làm hệ thống:

- 1.000 sinh viên thì **cùng lúc có bao nhiêu người** dùng? Cần bao nhiêu **băng thông** (đường truyền mạng), bao nhiêu **dung lượng lưu trữ**?
- Hệ thống gồm những phần nào, nối với nhau ra sao (**kiến trúc**)?
- Dựng thử được không, **đo** được không: chậm ở đâu, chịu được bao nhiêu người?
- Khi hỏng (máy chết, mất dữ liệu) thì **khôi phục** thế nào, mất bao lâu, mất bao nhiêu dữ liệu?

Phiếu đề chia thành 4 yêu cầu và 3 bài kiểm thử. Nhóm gọi bằng lời thường như sau:

Phần của đề	Nội dung	Ai lo chính
Yêu cầu 1: thiết kế	Thiết kế web, dữ liệu, video, đăng nhập; phân biệt **tổng tài khoản** với **số người dùng cùng lúc**	Đăng
Yêu cầu 2: tính tài nguyên	Tính băng thông, lưu trữ cho ít nhất 2 tình huống, ghi rõ mọi giả định	Đăng
Yêu cầu 3: triển khai	Dựng 1 web học liệu nhỏ có ít nhất 3 tài nguyên (video, PDF, bài nghe); thử 3 mức tải	Thành (web), Đăng (lab)
Yêu cầu 4: đo và đề xuất	Đo thời gian trả lời, số request mỗi giây, lỗi, CPU/RAM; chỉ ra điểm nghẽn; sao lưu; phân quyền	Vân Anh (đo), Đức (sao lưu, quyền)
Kiểm thử 1	Máy khách mở được trang và tải đúng học liệu	Vân Anh, Thành
Kiểm thử 2	Chạy 3 mức tải, ghi số liệu, điều kiện đo lặp lại được	Vân Anh
Kiểm thử 3	Dừng dịch vụ rồi khôi phục; kiểm tra học liệu và thời gian gián đoạn	Đức

Hồ sơ phải nộp (mục "Hồ sơ nộp chung" của phiếu đề): báo cáo nhóm 12–18 trang; sơ đồ và bảng tài nguyên/IP/quyền; cấu hình, script, mã nguồn; README hướng dẫn dựng lại; bảng kiểm thử có giờ chạy; dữ liệu đo gốc; video demo 5–7 phút; báo cáo cá nhân. Tất cả nằm trong thư mục ho-so-nop.

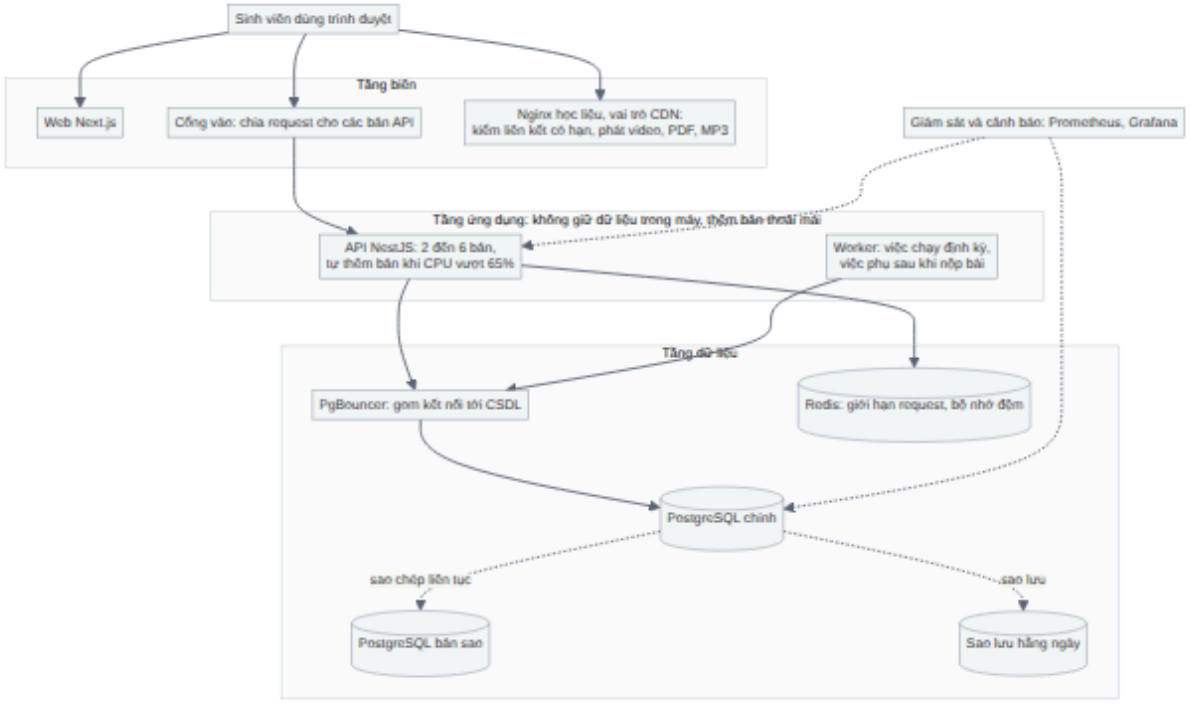
Buổi bảo vệ: mỗi người trình bày phần mình, **mở minh chứng** (tệp kết quả), **làm 1 thao tác hoặc sửa 1 thay đổi nhỏ** ngay trước mặt thầy, rồi **trả lời vấn đáp** (thầy có thể hỏi cả kiến trúc chung). Điểm cá nhân chiếm 50/100: 15 điểm phần kỹ thuật có bằng chứng, 15 điểm giải thích và thao tác trực tiếp, 20 điểm vấn đáp. Tài liệu này giúp bạn làm tốt cả 3 việc đó.

> Hanni chỉ là **hệ thống minh hoạ** (web học tiếng Trung mà Đăng viết từ trước). Thứ được chấm là cách nhóm thiết kế, đo, khôi phục, mở rộng, chứ không phải tính năng học tiếng Trung.

1.2 Hệ thống gồm những phần nào (ví như một quán ăn)

Hãy hình dung hệ thống như một quán ăn đông khách:

Phần trong hệ thống	Ví như	Làm gì
Trình duyệt của sinh viên	Khách hàng	Gửi yêu cầu, nhận kết quả, hiện lên màn hình
Web (Next.js, trang `/hoc-lieu`)	Thực đơn và bàn ăn	Phần giao diện sinh viên nhìn thấy, bấm vào
API (NestJS)	Bếp	Nhận yêu cầu ("cho tôi danh sách học liệu"), xử lý, trả kết quả. Chạy nhiều bản như nhiều đầu bếp
CSDL PostgreSQL	Kho nguyên liệu, sổ sách	Nơi giữ dữ liệu thật: tài khoản, tiến độ học, bài thi. Mất là mất thật
Redis	Bảng ghi chú dán tường	Ghi tạm những thứ cần đọc nhanh, mất cũng dựng lại được (bộ đếm giới hạn, bộ nhớ đệm)
PgBouncer	Người giữ cửa kho	Gom yêu cầu vào kho của nhiều đầu bếp, chỉ cho một số lượng cố định vào cùng lúc để kho không quá tải
Nginx học liệu (vai CDN)	Quầy phát đồ đóng gói sẵn	Phát thẳng video, PDF, MP3 cho khách, không bắt bếp phải làm. Khách phải có "phiếu" (liên kết có hạn)
Worker	Nhân viên hậu cần	Làm việc nền: nhắc học hằng ngày, xử lý việc phụ sau khi nộp bài
Giám sát (Prometheus, Grafana, Alertmanager)	Camera và chuông báo	Đo mọi thứ liên tục, vẽ biểu đồ, kêu khi có sự cố
Sao lưu	Bản photo sổ sách cất két	Mỗi ngày chép CSDL và học liệu ra chỗ khác để hỏng còn lấy lại được



Hình 1. Kiến trúc đề xuất cho 1.000 sinh viên (tệp gốc: ho-so-nop/so-do-va-bang/kien-truc-dich.png)

Ba tầng trong hình: tầng biên (web, cổng vào, Nginx học liệu) là chỗ khách đi vào; tầng ứng dụng (API, worker) là chỗ xử lý; tầng dữ liệu (PostgreSQL, Redis, sao lưu) là chỗ giữ dữ liệu. Ý quan trọng nhất: **tầng ứng dụng không giữ dữ liệu trong máy**, mọi thứ cần nhớ đều nằm ở CSDL hoặc Redis, nên muốn chịu nhiều người hơn chỉ cần **thêm bản API**, không phải sửa code.

1.3 Từ điển thuật ngữ (đọc kỹ, gặp lại ở khắp tài liệu)

Mạng và web

- **Request (yêu cầu)**: một lần trình duyệt hỏi máy chủ, ví dụ "cho tôi danh sách học liệu". Mỗi lần bấm nút có thể sinh ra vài request.
- **Response (trả lời)**: máy chủ đáp lại request, kèm một **mã trả về** (mã HTTP) cho biết kết quả.
- **API**: tập các "cửa" mà máy chủ mở ra để trình duyệt gọi vào, mỗi cửa là một **route** (đường dẫn), ví dụ `GET /api/resources` (lấy danh sách học liệu), `POST /api/resources/:id/link` (xin liên kết tải). `GET` là đọc, `POST` là gửi hoặc tạo mới.
- **Mã trả về HTTP** hay gặp trong dự án:

Mã	Nghĩa	Gặp ở đâu trong dự án
200	Thành công	Gọi API bình thường
401	Chưa đăng nhập	Khách xin liên kết video cần đăng nhập
403	Đã biết là ai nhưng không có quyền, hoặc chữ ký sai	Sửa chữ ký liên kết học liệu; người học gọi chức năng quản trị
404	Không có thứ này (hoặc giấu đi cho an toàn)	Gọi `/api/metrics` từ ngoài
410	Đã từng có nhưng hết hạn	Dùng liên kết học liệu đã quá 10 phút
429	Gửi quá nhiều, chờ chút rồi thử lại	Một tài khoản gọi quá 300 request trong 1 phút
503	Máy chủ đang quá tải hoặc chưa sẵn sàng, "thử lại sau"	Cơ chế từ chối khi quá tải của nhóm; API mất CSDL
5xx khác (500, 502…)	Lỗi máy chủ thật	Trong các lượt đo chính thức: 0 lỗi loại này

- **Băng thông**: lượng dữ liệu đi qua đường mạng mỗi giây, đơn vị **Mbps** (megabit mỗi giây). Chú ý: 8 bit = 1 byte, nên 8 Mbps = 1 MB mỗi giây. Video 540p của nhóm là 2 Mbps, tức mỗi giây xem tốn khoảng 0,25 MB.

Máy chủ

- **CPU, lõi (core)**: bộ xử lý. "0,25 lõi" nghĩa là được dùng một phần tư sức một lõi. Trong Kubernetes viết `250m` (250 phần nghìn lõi).
- **RAM**: bộ nhớ đang làm việc, viết `256Mi` (256 mebibyte, gần bằng 256 MB). Hết RAM thì máy phải dùng **swap** (lấy ổ đĩa làm RAM tạm), chậm hẳn đi; vì vậy lần đo nào máy bị swap thì nhóm bỏ, đo lại.
- **Tiến trình**: một chương trình đang chạy, ví dụ 1 bản API là 1 tiến trình Node.js.
- **Event loop (vòng xử lý của Node.js)**: Node.js xử lý mọi request bằng một vòng lặp duy nhất. Khi CPU quá bận, vòng này bị **trễ** (event loop lag): request phải xếp hàng chờ dù chưa cần tới CSDL.

Nhóm đo độ trễ này để biết API đang quá tải.

Docker và Kubernetes

- **Docker, container, image**: Docker đóng gói một chương trình cùng mọi thứ nó cần vào một **image** (như hộp cơm đóng sẵn). Chạy image lên thành **container** (hộp cơm đang được ăn). Nhờ vậy máy nào có Docker cũng chạy được giống hệt nhau. Mọi công cụ của lab (k6, ffmpeg, kubectl) đều chạy trong container nên máy không phải cài gì thêm.
- **Kubernetes**: phần mềm **quản lý nhiều container** trên nhiều máy: tự bật lại container chết, tự thêm bản khi đông, chia request đều. **k3s** là bản Kubernetes nhẹ, nhóm chạy nó ngay trong Docker để dựng lab trên 1 máy.
- **Cụm (cluster)**: tập các máy Kubernetes quản lý. **Node**: mỗi máy trong cụm. Lab có 2 node: `k3s-server` (172.25.0.2) và `k3s-agent` (172.25.0.3).
- **Pod**: đơn vị nhỏ nhất Kubernetes chạy, thường là 1 container. 1 bản API = 1 pod.
- **Deployment**: bản mô tả "tôi muốn chạy N pod giống nhau từ image X". Kubernetes giữ đúng N pod, chết cái nào tạo lại cái đó.
- **Bản (replica)**: mỗi pod giống nhau trong một Deployment. "API chạy 6 bản" là 6 pod API cùng phục vụ.
- **Service**: một địa chỉ cố định đứng trước nhiều pod, chia request cho các pod đó. Pod chết, đổi địa chỉ, Service vẫn giữ nguyên.
- **Ingress (cổng vào)**: cửa chính từ ngoài vào cụm, xem đường dẫn để đưa request tới đúng Service (`api.hanni.localhost` tới API, `media.hanni.localhost` tới Nginx học liệu). Lab dùng Traefik làm cổng vào, mở ở `127.0.0.1:18080`.
- **Namespace**: ngăn chia trong cụm. Ứng dụng nằm ở `hanni`, giám sát nằm ở `observability`.
- **Manifest (tệp YAML)**: tệp mô tả thứ muốn chạy (Deployment, Service...). Nhóm sửa manifest rồi **apply** (nạp) vào cụm.
- **kubectl**: lệnh điều khiển cụm. Trong lab gọi qua `scripts/kubectl` (đã nối sẵn vào lab). Vài lệnh hay dùng: `scripts/kubectl -n hanni get pods` (xem các pod), `get hpa` (xem tự mở rộng), `logs <tên pod>` (xem nhật ký).
- **HPA (tự mở rộng)**: Horizontal Pod Autoscaler, cứ 15 giây đo CPU trung bình các bản API; vượt ngưỡng (65% mức CPU yêu cầu) thì thêm bản, thấp lâu thì bớt, trong khoảng 2–6 bản.
- **Probe (kiểm tra sức khoẻ)**: Kubernetes định kỳ gọi thử vào pod. **Readiness** hỏi "sẵn sàng nhận khách chưa?", chưa thì tạm không chia request cho pod đó. **Liveness** hỏi "còn sống không?", nhiều lần không trả lời thì khởi động lại pod.

Dữ liệu

- **CSDL (cơ sở dữ liệu) PostgreSQL**: nơi lưu dữ liệu dạng bảng (bảng `User`, `ReviewLog`...), mỗi dòng là một bản ghi.
- **Kết nối CSDL, pool**: mỗi lần API nói chuyện với CSDL phải dùng một **kết nối**. Pool là "hồ" kết nối mở sẵn để dùng lại. CSDL chỉ chịu được một số kết nối nhất định; hết kết nối thì request phải **xếp hàng chờ**, đây là điểm nghẽn đầu tiên nhóm đo được.
- **Bản sao (replica) CSDL**: một CSDL thứ hai liên tục chép theo CSDL chính, dùng để đọc bớt và để dự phòng khi CSDL chính hỏng.
- **WAL (nhật ký thay đổi)**: PostgreSQL ghi mọi thay đổi vào nhật ký trước. Nhờ nhật ký này mà bản sao chép theo được, và có thể khôi phục về **đúng một thời điểm** trước khi xoá nhầm.

- **Redis**: kho dữ liệu cực nhanh nằm trong RAM, dùng cho thứ mất cũng không sao: bộ đếm giới hạn request, bộ nhớ đệm.
- **Bộ nhớ đệm (cache)**: lưu tạm kết quả đã tính để lần sau trả ngay, không phải tính lại. Nhóm lưu tạm "khung lộ trình học" 60 giây.

Đo hiệu năng

- **Tải**: lượng người và request đổ vào hệ thống. Đo bằng **request/giây** hoặc **lượt/giây** (1 lượt mở bài = 5 request đọc).
- **Người dùng cùng lúc (phiên đồng thời)**: số người đang thật sự dùng ở cùng một thời điểm. 1.000 tài khoản không có nghĩa 1.000 người cùng lúc: tự học rải rác chỉ khoảng **63 người**, cả lớp xem video 300, thi thử đồng loạt 600.
- **Định luật Little**: số người cùng lúc = số người bắt đầu mỗi phút × số phút mỗi người ở lại. Ví như quán ăn: mỗi phút có 2,5 khách vào, mỗi khách ngồi 25 phút, thì lúc nào cũng có khoảng 63 khách trong quán.
- **Thời gian trả lời** (độ trễ): từ lúc gửi request tới lúc nhận trả lời, đơn vị **ms** (mili giây, 1.000 ms = 1 giây).
- **p50, p95, p99 (phân vị)**: xếp thời gian trả lời của mọi request từ nhanh tới chậm. **p95 = 40 ms** nghĩa là 95% request nhanh hơn 40 ms, 5% chậm hơn. Nhóm dùng p95 vì **trung bình** bị số đông nhanh che mất những người phải chờ lâu.
- **Thông lượng**: số request trả lời được mỗi giây.
- **Tỉ lệ lỗi**: phần trăm request bị lỗi.
- **SLO (ngưỡng đạt)**: mức chất lượng nhóm tự đặt ra: **95% lượt đọc dưới 300 ms, 95% lượt ghi dưới 500 ms, lỗi dưới 0,5%**. Đo mà vượt ngưỡng là "không đạt".
- **Điểm gãy**: mức tải cao nhất còn đạt SLO; tăng quá đó là chậm vọt. Lab đo được khoảng 40 lượt/giây (khoảng 200 request/giây).
- **Điểm nghẽn (nút thắt)**: bộ phận bị đầy đầu tiên khiến cả hệ thống chậm. Như con đường: chỗ hẹp nhất quyết định xe chạy được bao nhiêu.
- **Từ chối khi quá tải**: khi quá tải, API trả lời ngay mã 503 "thử lại sau" cho một phần lượt đọc, thay vì nhận hết rồi bắt tất cả chờ 5 giây. Người được phục vụ thì nhanh, người bị từ chối thì biết để thử lại.
- **k6**: công cụ **giả lập nhiều người dùng** gửi request đều đặn rồi đo thời gian trả lời. Nhóm viết **kịch bản** k6 bằng JavaScript.
- **Lab**: môi trường đo riêng của nhóm (cụm k3s 2 node trong Docker, dữ liệu giả 1.000 tài khoản), tách hẳn web thật để đo không ảnh hưởng người dùng.
- **GitHub Actions**: dịch vụ của GitHub cho mượn máy ảo chạy lệnh tự động. Laptop của nhóm không đủ RAM nên **số đo chính thức** chạy trên máy ảo GitHub (4 nhân, 16 GB) ngày 09/11/2026; máy ảo dựng lại lab từ đầu theo README rồi mới đo.

Giám sát và sao lưu

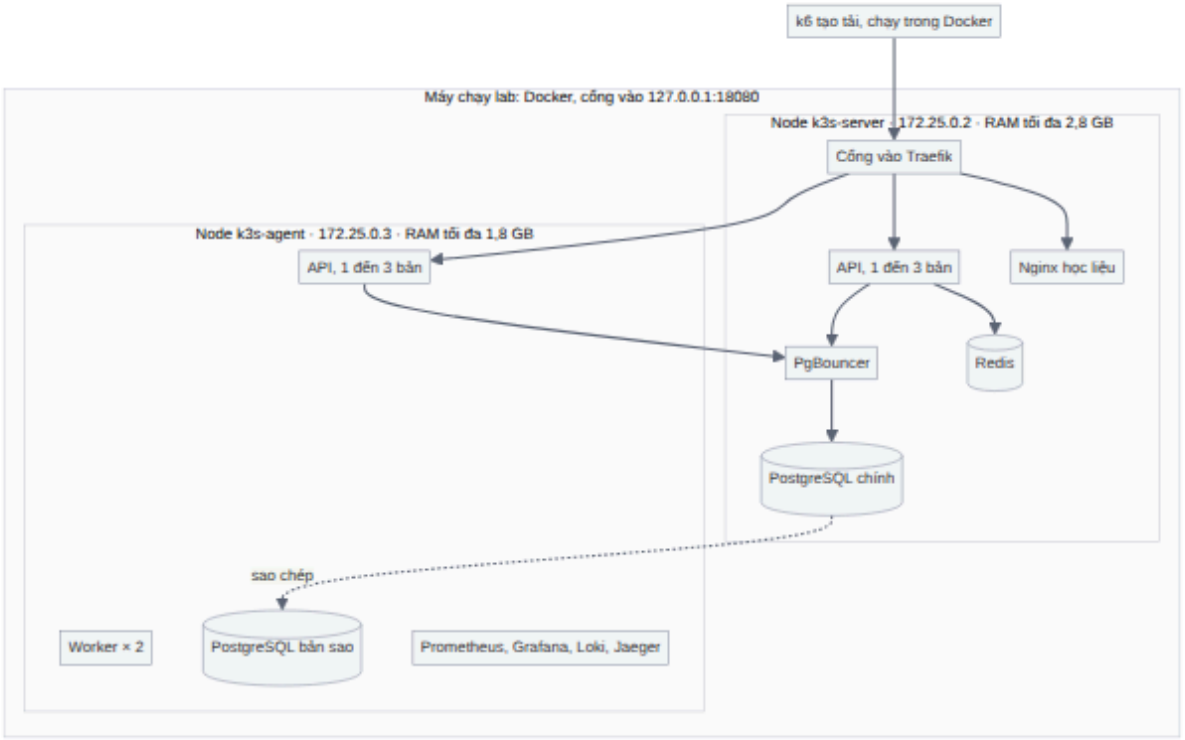
- **Prometheus**: cứ vài giây hỏi số liệu của từng thành phần (API, CSDL, Redis) rồi lưu lại theo thời gian.
- **Grafana**: vẽ số liệu của Prometheus thành biểu đồ. Nhóm có 3 bảng: "Hanni: SLO và tín hiệu vàng", "Hanni: Mở rộng", "Hanni: Dữ liệu" (mục 1.15).
- **Alertmanager**: gửi cảnh báo khi một luật bị vi phạm, ví dụ "từ ngoài gọi API không được quá 30 giây".

- **Blackbox (dò từ ngoài)**: một chương trình đứng ngoài gọi thử API mỗi vài giây như một sinh viên thật, để biết sinh viên có vào được không.
- **Loki, Jaeger**: Loki gom nhật ký (log) của mọi pod; Jaeger lần theo đường đi của một request qua các phần.
- **Sao lưu (backup)**: chép dữ liệu ra một bản riêng để khi hỏng lấy lại được. **Khôi phục (restore)**: đưa bản sao lưu trở lại.
- **RPO**: lượng dữ liệu tối đa có thể mất, tính từ lần sao lưu gần nhất. Sao lưu mỗi ngày thì RPO tối đa 1 ngày.
- **RTO**: thời gian từ lúc hỏng tới lúc dùng lại được. Lab đo được RTO 52 giây khi xoá hẳn CSDL.
- **Mã băm SHA-256**: "dấu vân tay" của tệp, một chuỗi 64 ký tự tính từ nội dung. Sai một byte là chuỗi khác hẳn, nên so mã băm biết tệp tải về hay khôi phục ra có đúng y nguyên không.

Công cụ dựng lab

- **make, Makefile**: `make up` là chạy một nhóm lệnh đã viết sẵn trong tệp `Makefile`. Nhóm có `make up`, `make images`, `make deploy`, `make seed`, `make observability`, `make status`, `make stop`, `make start`, `make down`.
- **YAML**: định dạng các tệp cấu hình. Quy tắc: `khoá: giá trị`, **thụt lề bằng dấu cách** thể hiện cấp (sai thụt lề là hỏng tệp), dòng bắt đầu bằng `#` là ghi chú, chuỗi số nên để trong ngoặc kép như `"600"`.

1.4 Lab của nhóm trông như thế nào



Hình 2. Sơ đồ lab: 2 node, địa chỉ IP và các thành phần (tệp gốc: ho-so-nop/so-do-va-bang/so-do-lab.png)

Thành phần	Địa chỉ	Ghi chú
Node k3s-server	172.25.0.2	RAM tối đa 2,8 GB

Thành phần	Địa chỉ	Ghi chú
Node k3s-agent	172.25.0.3	RAM tối đa 1,8 GB
Cổng vào (ingress)	127.0.0.1:18080	Mọi địa chỉ dưới đây đi qua cổng này
API	http://api.hanni.localhost:18080/api/health	
Grafana	http://grafana.hanni.localhost:18080	Tài khoản `admin`, mật khẩu lấy bằng lệnh ở README
Prometheus	http://prometheus.hanni.localhost:18080	
Alertmanager	http://alertmanager.hanni.localhost:18080	
Web	http://hanni.localhost:3000	Chạy riêng bằng `npx next dev`

1.5 Repo này có gì, phần của mình ở đâu

Đường dẫn	Nội dung
ho-so-nop/bao-cao/	Báo cáo nhóm (.docx, .pdf), 4 báo cáo cá nhân theo mẫu của thầy
ho-so-nop/so-do-va-bang/	2 sơ đồ, bảng tài nguyên/IP/quyền kèm bảng tính tài nguyên
ho-so-nop/cau-hinh-ma-nguon/hanni-server/infra/	**Toàn bộ cấu hình lab**: k8s/ (Kubernetes), k6/ (kịch bản tải), scripts/ (script kiểm thử, sao lưu), observability/ (giám sát), media/ (học liệu)
ho-so-nop/cau-hinh-ma-nguon/hanni-server/src/	Mã API nhóm viết cho đề: học liệu, số liệu đo, giới hạn request, từ chối khi quá tải
ho-so-nop/cau-hinh-ma-nguon/hanni-client/	Trang web học liệu và test tự động
ho-so-nop/data/raw/	Dữ liệu đo gốc, không sửa tay. `gha-09-11/` là lượt đo chính thức
ho-so-nop/evidence/	Minh chứng: ảnh chụp, nhật ký, danh sách mã băm. Danh mục ở `evidence/README.md`
ho-so-nop/bang-kiem-thu.xlsx	Bảng kiểm thử 62 dòng có giờ chạy
ho-so-nop/video-demo.mp4	Video demo 6 phút
Hanni-RIPT1304-Phan-cong.xlsx	Sheet phân công: Tổng quan (vai trò, tiến độ) và Công việc (ai làm gì, minh chứng ở đâu)

Mã nguồn đầy đủ (chạy được) ở 2 repo: hanni-server nhánh feat/scaling và hanni-client nhánh feat/hoc-lieu. Thư mục `cau-hinh-ma-nguon/` chỉ là bản trích các tệp nhóm viết cho đề.

1.6 Dựng lab trên máy của bạn

Cần: đã cài đủ công cụ theo mục 0.4, còn trống khoảng 5 GB RAM và 8 GB đĩa. Mỗi lệnh dưới đây chạy xong mới gõ lệnh tiếp theo.

```
cd ~
git clone -b feat/scaling https://github.com/DANG-PH/hanni-server.git
cd hanni-server/infra       # MỌI lệnh make và scripts/ đều chạy trong thư mục này
```

Lệnh	Làm gì	Mất bao lâu	Thấy gì là đúng
`make up`	Dựng 3 học liệu mẫu (lần đầu), bật 2 container node, chờ 2 node sẵn sàng, sinh mật khẩu ngẫu nhiên	25 giây; lần đầu lâu hơn vì phải tải image	Bảng 2 node đều `Ready`, các dòng `secret/... created`
`make images`	Build image API từ mã nguồn, nạp vào 2 node	87 giây; lần đầu 5–15 phút	Tự trả lại dấu nhắc, không có chữ `ERROR`
`make deploy`	Nạp mọi tệp trong `k8s/` vào cụm, chờ Postgres, bản sao, PgBouncer sẵn sàng	33 giây	Dòng cuối `deployment "hanni-pgbouncer" successfully rolled out`
`make seed`	Tạo bảng, nạp từ vựng, tạo 1.000 tài khoản giả, khởi động lại API	53 giây	Dòng cuối `deployment "hanni-api" successfully rolled out`
`make observability`	Bật Prometheus, Alertmanager, Grafana, Loki, Jaeger	2 giây, nhưng pod cần thêm 1–2 phút	Các dòng `... created`
`make status`	Xem trạng thái toàn bộ	tức thì	Xem giải thích dưới đây

Thời gian đo trên máy của Đăng ngày 07/11 khi đã có sẵn image (nhật ký đầy đủ: evidence/dung-lai/dung-lai-tu-readme.log). Đoạn cuối của `make up` trên máy đó:

```
kubectl get nodes
NAME         STATUS   ROLES                  AGE   VERSION
k3s-agent    Ready    <none>                 3s    v1.31.6+k3s1
k3s-server   Ready    control-plane,master   13s   v1.31.6+k3s1
./scripts/secrets.sh
secret/hanni-secrets created
secret/grafana-admin created
```

Đầu ra của `make status` (rút gọn) và cách đọc:

```
pod/hanni-postgres-0                       2/2     Running       0          86s
pod/hanni-seed-t94vt                       0/1     Completed     0          48s
pod/hanni-worker-567b6c9597-bb5l8          1/1     Running       0          86s
NAME                                              REFERENCE              TARGETS              MINPODS
MAXPODS   REPLICAS
horizontalpodautoscaler.autoscaling/hanni-api   Deployment/hanni-api   cpu: <unknown>/65%   2
6         2
grafana-5c46b9bc-tdgnx                0/1     ContainerCreating   0          0s
```

- Pod `Running` là đang chạy, `Completed` là việc chạy 1 lần đã xong (tạo bảng, nạp dữ liệu). Cả hai đều đúng.
- HPA ghi `cpu: <unknown>/65%` ngay sau khi dựng là **bình thường**: khoảng 1 phút sau mới có số đo CPU.
- Pod giám sát `ContainerCreating` ngay sau `make observability` cũng bình thường: chờ 1–2 phút rồi gõ lại `make status`, phải thành `Running`.

Kiểm tra API sống: `curl http://api.hanni.localhost:18080/api/health`, thấy `{"status":"ok","db":"up","redis":"up",...}`. Mở trình duyệt vào http://grafana.hanni.localhost:18080 thấy trang đăng nhập Grafana là cổng vào đã thông.

Tạm tắt cho nhẹ máy: `make stop`, bật lại: `make start`, xoá sạch: `make down`. Máy yếu không dựng nổi thì nhờ Đăng chạy trên GitHub Actions (đẩy nhánh lên `do-lab` là máy ảo tự dựng và đo).

1.7 Quy tắc chung khi sửa cấu hình

1. Mở tệp cần sửa trong `hanni-server/infra` (đường dẫn ở bảng "Sửa nhanh" của phần riêng bạn), sửa đúng dòng, giữ nguyên thụt lề.
2. Nạp lại vào cụm: `make deploy`.
3. Nếu đã sửa `k8s/app/config.yaml` hoặc `k8s/media-manifest.json` thì chạy thêm `scripts/kubectl -n hanni rollout restart deployment/hanni-api`, vì API chỉ đọc 2 tệp này lúc khởi động.
4. Nếu đã sửa luật cảnh báo trong `observability/` thì chạy `make observability` rồi `scripts/kubectl -n observability rollout restart deployment/prometheus`.
5. **Kiểm tra lại** bằng lệnh hoặc trang ghi ở cột "Kiểm tra" và nói được kết quả thay đổi thế nào.

> Khi thầy bảo "đổi X rồi chạy lại", hãy nói to 3 ý: (1) X nằm ở tệp nào, dòng nào; (2) đổi xong thì hệ thống sẽ thay đổi gì; (3) lệnh nào kiểm tra được điều đó. Làm như vậy là ăn trọn điểm "giải thích và thao tác trực tiếp".

1.8 Lỗi hay gặp khi dựng lab và cách xử lý

Cách tự tìm lỗi trước khi hỏi: (1) `make status` xem pod nào không ở trạng thái `Running`; (2) `scripts/kubectl -n hanni describe pod <tên pod>`, đọc mục **Events** ở cuối (Kubernetes ghi lý do bằng tiếng Anh, ví dụ thiếu RAM, thiếu image); (3) `scripts/kubectl -n hanni logs <tên pod>` xem nhật ký của chính chương trình.

Hiện tượng	Nguyên nhân	Cách xử lý
`permission denied ... docker.sock` khi chạy `make`	Tài khoản chưa có quyền dùng Docker	`sudo usermod -aG docker $USER`, đăng xuất rồi đăng nhập lại (hoặc chạy lệnh bằng `sudo`)
`port is already allocated` khi `make up`	Cổng 18080 đã có chương trình khác dùng	Tắt chương trình đó, hoặc đổi số 18080 ở dòng `127.0.0.1:18080:80` trong `compose.yaml` (mọi địa chỉ đổi theo)
Lần đầu `make up`, `make images` rất lâu	Phải tải image k3s, PostgreSQL, Node.js…	Bình thường, 10–20 phút lần đầu, các lần sau nhanh
Pod API báo `ErrImageNeverPull`	Chưa chạy `make images`: image API build tại chỗ, manifest đặt `imagePullPolicy: Never` (không tải từ mạng)	`make images` rồi `make deploy`
Pod `Pending` hoặc `OOMKilled`, máy rất chậm	Thiếu RAM: 2 node được dùng tới 2,8 GB + 1,8 GB	Đóng bớt ứng dụng; chỉ thử API thì bỏ `make observability`; máy dưới 8 GB RAM thì nhờ Đăng chạy trên GitHub Actions
`kubectl exec` báo lỗi chứng chỉ sau khi khởi động lại máy	Docker đổi IP của node	`compose.yaml` đã cố định IP; nếu vẫn gặp: `make down` rồi dựng lại

Hiện tượng	Nguyên nhân	Cách xử lý
Đăng nhập trên web xong vẫn như khách	Mở `localhost:3000` thay vì `hanni.localhost:3000`, trình duyệt không gửi cookie sang API	Mở đúng `http://hanni.localhost:3000`
k6 báo vài % lỗi ở đầu lần đo, mức cao hơn lại không lỗi	Đo đúng lúc Kubernetes đang thêm bản API	Chờ đủ bản rồi mới đo (`scripts/cho-du-ban.sh`; `chay-kt2.sh` đã tự làm)
Sau khi chạy `failover.sh` hoặc `do-phan-manh.sh` lab chạy lạ	`failover.sh` cố ý tắt CSDL chính và đưa bản sao lên thay; `do-phan-manh.sh` chia hẳn bảng ReviewLog. Lab không còn như lúc đầu	`make down` rồi dựng lại từ đầu trước khi đo tiếp

1.9 Kubernetes cho người chưa dùng bao giờ

Kubernetes giải quyết việc gì

Không có Kubernetes, muốn chạy API ta mở máy chủ, gõ lệnh chạy chương trình. API chết thì phải có người bật lại; đông khách thì phải tự mở thêm bản và tự chia request cho các bản. Kubernetes làm hộ những việc đó theo một nguyên tắc duy nhất: **bạn viết ra trạng thái mong muốn, Kubernetes liên tục so với trạng thái đang có và tự sửa cho khớp**. Ví dụ bạn viết "cần ít nhất 2 bản API". Một bản chết, đang có 1, khác mong muốn, Kubernetes tự tạo bản mới. Ví như quản lý quán ăn có tờ giấy ghi "ca này cần 2 đầu bếp": 1 người nghỉ thì quản lý gọi người khác vào, không cần chủ quán dặn.

"Trạng thái mong muốn" được viết trong các **tệp YAML** (mục 1.10). Lệnh `make deploy` nạp các tệp đó vào cụm; từ lúc đó Kubernetes tự giữ.

> **Web Hanni thật không chạy trên Kubernetes.** Trên VPS (2 vCPU, 3,8 GB RAM, vài người dùng thật), API chạy bằng **pm2** (trình giữ cho chương trình Node.js luôn chạy, chết thì tự bật lại), PostgreSQL, Redis, Nginx chạy bằng **Docker**. Kubernetes chỉ có trong **lab** để thử phương án cho 1.000 sinh viên: tự mở rộng, tự thay bản chết, rải bản ra nhiều máy, chặn mạng. Nếu thầy hỏi "sao không dùng Kubernetes luôn": với 1 máy và vài người dùng thì pm2 + Docker đủ và nhẹ hơn (riêng k3s đã tốn khoảng 650 MB RAM); khi cần nhiều máy, nhiều bản API cho 1.000 sinh viên mới cần tới Kubernetes. Đây chính là "kiến trúc hiện tại" và "kiến trúc đích" trong báo cáo nhóm.

Một request đi qua những gì trong lab

```
Trình duyệt gọi http://api.hanni.localhost:18080/api/resources
   │  cổng 18080 trên máy bạn, Docker chuyển vào node k3s-server (compose.yaml)
   ▼
Ingress (Traefik)          xem tên miền "api.hanni.localhost" → chuyển cho Service hanni-api
(k8s/app/ingress.yaml)
   ▼
Service hanni-api          chọn 1 trong các pod mang nhãn app: hanni-api, chia đều
(k8s/app/api.yaml)
   ▼
Pod API (1 trong 2-6 bản)  chạy NestJS, xử lý request
   ├──► Service hanni-pgbouncer → Pod PgBouncer → Pod hanni-postgres-0 (CSDL)  (k8s/data/)
   └──► Service hanni-redis → Pod Redis                                        (k8s/data/redis.yaml)
```

Video, PDF, MP3 đi đường khác: `media.hanni.localhost` → Ingress → Service `hanni-media` → Pod Nginx (`k8s/app/media.yaml`), không qua API.

Các loại đối tượng trong lab

Mỗi khối trong tệp YAML khai 1 **đối tượng**, dòng `kind:` cho biết loại. Lab dùng các loại sau (đếm trong `k8s/` và `observability/`):

Loại (`kind`)	Ví như	Trong lab	Tệp
Namespace	Khu riêng trong quán	`hanni` (ứng dụng), `observability` (giám sát)	`k8s/namespace.yaml`
Deployment	Tờ giấy "cần N đầu bếp giống nhau"	API, worker, PgBouncer, Redis, Nginx học liệu, Grafana… (15 cái)	`k8s/app/api.yaml` …
ReplicaSet	Ca làm cụ thể theo tờ giấy đó	Deployment tự tạo, bạn không viết. Mỗi lần đổi cấu hình API sinh ReplicaSet mới, bản cũ được thay dần	(tự sinh)
Pod	1 đầu bếp đang làm	Tên dạng `hanni-api-5456c9c9cb-5xzh5` = tên Deployment + mã ReplicaSet + mã riêng của pod	(tự sinh)
StatefulSet	Thủ kho có tên và chìa khoá riêng	Postgres chính và bản sao: tên cố định `hanni-postgres-0`, mỗi pod có ổ đĩa riêng, pod chết tạo lại vẫn gắn đúng ổ cũ	`k8s/data/postgres*.yaml`
PersistentVolumeClaim	Phiếu xin 1 tủ chứa đồ	Ổ đĩa cho Postgres (2 GB), thư mục sao lưu	`k8s/data/`, `k8s/backup/`
Service	Số điện thoại cố định của 1 bộ phận	`hanni-api`, `hanni-pgbouncer`, `hanni-redis` … Pod đổi, Service vẫn giữ tên	trong từng tệp
Ingress	Lễ tân ở cửa	Tên miền nào đi tới Service nào	`k8s/app/ingress.yaml`
ConfigMap	Bảng nội quy dán trong bếp	Biến môi trường của API (`hanni-env`), cấu hình Nginx, danh sách học liệu, script sao lưu	`k8s/app/config.yaml` …
Secret	Két giữ chìa khoá	Mật khẩu CSDL, khoá ký liên kết. Sinh ngẫu nhiên lúc `make up`, không nằm trong tệp nào	`scripts/secrets.sh`
HorizontalPodAutoscaler (HPA)	Quản lý gọi thêm đầu bếp khi đông	Tự đổi số bản API 2–6 theo CPU	`k8s/app/api.yaml`

Loại (`kind`)	Ví như	Trong lab	Tệp
PodDisruptionBudget (PDB)	"Luôn phải có ít nhất 1 người trong bếp"	Khi bảo trì node, luôn còn ít nhất 1 bản API	`k8s/app/api.yaml`
Job, **CronJob**	Việc làm 1 lần / việc làm theo lịch	Job tạo bảng, nạp dữ liệu (`make seed`); CronJob sao lưu 02:00	`k8s/app/jobs.yaml` , `k8s/backup/cronjob.yaml`
NetworkPolicy	Quy định ai được vào phòng nào	Chặn mọi đường mạng trong cụm, chỉ mở đường cần thiết	`k8s/policy/network-policy.yaml`

Tự mở rộng (HPA): làm được gì, không làm được gì

Có, lab **tự mở rộng thật**: HPA trong `k8s/app/api.yaml` cứ 15 giây đo CPU trung bình các bản API, rồi tính số bản cần theo công thức của Kubernetes:

> **Số bản cần = làm tròn lên (số bản hiện có × CPU hiện tại ÷ CPU mục tiêu)**. Mục tiêu là 65% mức CPU yêu cầu (0,25 lõi). Ví dụ 2 bản đang dùng trung bình 130%: 2 × 130 ÷ 65 = 4, Kubernetes nâng lên 4 bản. Kết quả luôn bị giữ trong khoảng `minReplicas` 2 tới `maxReplicas` 6.

Đo thật trong lượt chính thức (tăng tải dần): 4 bản (giây 0) → 3 bản (giây 45) → 5 bản (giây 90) → 6 bản (giây 120). Mỗi lần đổi, Kubernetes ghi 1 **sự kiện** (xem bằng `scripts/kubectl -n hanni get events`), đây là dòng thật của lượt đo:

```
24m  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 2; reason: Current
number of replicas below Spec.MinReplicas
22m  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 3; reason: cpu resource
utilization (percentage of request) above target
8m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 5; reason: All metrics
below target
7m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 4; reason: All metrics
below target
6m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 3; reason: All metrics
below target
5m44s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 5; reason: cpu
resource utilization (percentage of request) above target
5m14s  Normal  SuccessfulRescale  horizontalpodautoscaler/hanni-api  New size: 6; reason: cpu
resource utilization (percentage of request) above target
```

Dòng 1: thêm lên 3 bản vì "CPU vượt mục tiêu". Dòng 2: bớt về 5 bản vì "mọi chỉ số dưới mục tiêu" (lúc hết tải).

HPA không làm (thầy hay hỏi để xem bạn có hiểu giới hạn không):

- **Không thêm máy.** HPA chỉ thêm pod trên các node đang có. Thêm máy là việc của bộ tự thêm node (cluster autoscaler) trên cloud, lab không có (2 node chạy chung 1 máy).
- **Không mở rộng CSDL, Redis.** Chỉ API được tự mở rộng vì API không giữ dữ liệu. Postgres có 1 bản chính + 1 bản sao cố định; muốn hơn phải thiết kế lại.
- **Không kịp với đợt dồn 30 giây** (cả lớp nộp bài): đo 15 giây 1 lần, bản mới cần vài chục giây mới sẵn sàng. Trước giờ thi phải tăng sẵn `minReplicas` .

- **Không cứu được khi CSDL là điểm nghẽn**: thêm bản API chỉ làm thêm người xếp hàng chờ CSDL.

1.10 Đọc một tệp YAML của Kubernetes

Mọi đối tượng đều có 4 phần:

Dòng	Ý nghĩa
`apiVersion:`	Phiên bản cách viết của loại đối tượng này (`apps/v1` , `v1` ...). Chép đúng như mẫu, không phải nghĩ
`kind:`	Loại đối tượng (bảng ở mục 1.9)
`metadata:`	Tên (`name`), nhãn (`labels`), ghi chú cho công cụ khác (`annotations`)
`spec:`	**Phần mong muốn**: phần bạn đọc và sửa nhiều nhất

Quy tắc chữ: thụt lề bằng **dấu cách** thể hiện cấp (con nằm trong cha); dòng bắt đầu bằng `-` là 1 phần tử của danh sách; `#` là ghi chú; `---` ngăn 2 đối tượng trong cùng 1 tệp. Muốn biết tệp có những gì: `grep -n "kind:" k8s/app/api.yaml` (in ra Deployment, Service, HPA, PDB).

Nhãn (labels) và bộ chọn (selector): đây là cách các đối tượng tìm thấy nhau. Pod API mang nhãn `app: hanni-api` . Deployment quản lý những pod có nhãn đó, Service gửi request tới những pod có nhãn đó, HPA, PDB, NetworkPolicy cũng chọn theo nhãn đó. Sửa nhãn ở một chỗ mà quên chỗ kia thì Service "không thấy" pod nào, API mất kết nối dù pod vẫn chạy.

Đọc thử phần đầu tệp API (Deployment):

hanni-server/infra/k8s/app/api.yaml, dòng 1–46 (mở trên GitHub):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hanni-api
  labels:
    app: hanni-api
spec:
  # Không khai replicas: HPA giữ số bản, khai ở đây thì mỗi lần apply sẽ kéo số bản về lại giá trị này.
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: hanni-api
  template:
    metadata:
      labels:
        app: hanni-api
        tier: backend
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8000"
        prometheus.io/path: /api/metrics
    spec:
      terminationGracePeriodSeconds: 40
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: hanni-api
      containers:
        - name: api
          image: hanni-api:lab
          imagePullPolicy: Never
          ports:
            - name: http
              containerPort: 8000
          envFrom:
            - configMapRef:
                name: hanni-env
            - secretRef:
                name: hanni-secrets
```

Dòng	Đọc là
1–6	Một Deployment tên `hanni-api`, gắn nhãn `app: hanni-api`
8	Ghi chú: cố ý **không** khai số bản ở đây, vì HPA giữ số bản; khai ở đây thì mỗi lần `make deploy` số bản bị kéo về
9–13	Cách thay phiên bản mới: thêm tối đa 1 bản mới trước (`maxSurge: 1`), không tắt bản cũ nào khi bản mới chưa sẵn sàng (`maxUnavailable: 0`), nên đổi cấu hình không làm gián đoạn
14–16	Deployment quản lý các pod có nhãn `app: hanni-api`

Dòng	Đọc là
17–25	**Khuôn** để tạo từng pod: nhãn của pod, và ghi chú cho Prometheus biết lấy số liệu ở cổng 8000, đường `/api/metrics`
27	Khi tắt, cho pod 40 giây làm nốt request đang dở
28–34	Rải các bản API đều lên 2 node, để 1 node chết vẫn còn bản ở node kia
35–41	Container `api` chạy image `hanni-api:lab` (build tại chỗ bằng `make images`, nên không tải từ mạng: `Never`), nghe cổng 8000
42–46	Nạp biến môi trường từ ConfigMap `hanni-env` (tệp `k8s/app/config.yaml`) và Secret `hanni-secrets` (mật khẩu)

Phần còn lại của tệp (tài nguyên CPU/RAM, kiểm tra sức khoẻ, Service, HPA, PDB) được giải thích trong tài liệu của Đăng, phần 5.

ConfigMap là nơi đặt cấu hình dạng `TÊN: giá trị`, API đọc thành biến môi trường lúc khởi động (vì vậy sửa xong phải khởi động lại API):

hanni-server/infra/k8s/app/config.yaml, dòng 1–11 (mở trên GitHub):

```
   1 | apiVersion: v1
   2 | kind: ConfigMap
   3 | metadata:
   4 |   name: hanni-env
   5 | data:
   6 |   NODE_ENV: production
   7 |   PORT: "8000"
   8 |   API_PREFIX: /api
   9 |   APP_URL: http://api.hanni.localhost:18080
  10 |   # Client chạy ở hanni.localhost cho cùng site với API, nếu không trình duyệt không nhận
cookie đăng nhập.
  11 |   FRONTEND_URL: http://hanni.localhost:3000
```

Giá trị số hay `true` / `false` phải để trong ngoặc kép (`"8000"`, `"false"`) vì biến môi trường luôn là chữ.

Kustomization là danh sách các tệp mà `make deploy` nạp vào cụm. Tệp mới không có tên ở đây thì không bao giờ được nạp:

hanni-server/infra/k8s/kustomization.yaml, dòng 5–29 (mở trên GitHub):

```
resources:
  - namespace.yaml
  - data/postgres.yaml
  - data/postgres-replica.yaml
  - data/pgbouncer.yaml
  - data/redis.yaml
  - app/config.yaml
  - app/api.yaml
  - app/worker.yaml
  - app/media.yaml
  - app/ingress.yaml
  - policy/network-policy.yaml
  - backup/cronjob.yaml

configMapGenerator:
  - name: hanni-backup-script
    files:
      - backup/backup.sh
  - name: hanni-resources
    files:
      - manifest.json=media-manifest.json

generatorOptions:
  disableNameSuffixHash: true

```

Phần `configMapGenerator` biến `backup.sh` và `media-manifest.json` thành ConfigMap. Đó là lý do sửa danh sách học liệu chỉ cần `make deploy` rồi khởi động lại API, không phải build lại image.

1.11 Đọc một script bash

Các tệp `.sh` trong `scripts/` là **script bash**: một chuỗi lệnh dòng lệnh viết sẵn để khỏi gõ tay. **Mẹo quan trọng nhất**: mọi script của nhóm đều ghi **mục đích và cách gọi ở 2–8 dòng ghi chú đầu tệp**, đọc chỗ đó trước. Nhiều script gọi không kèm tham số sẽ in hướng dẫn (ví dụ `scripts/chaos.sh`).

Đọc thử 1 script ngắn, đủ các cú pháp hay gặp:

hanni-server/infra/scripts/do-gian-doan.sh, dòng 1–25 (mở trên GitHub):

```
#!/usr/bin/env bash
# Gọi API mỗi giây, ghi mã trả về vào CSV. Dừng bằng Ctrl+C thì in tổng thời gian gián đoạn dài nhất.
#   ./do-gian-doan.sh ket-qua/dung-api.csv
set -uo pipefail

out=${1:?Cần đường dẫn tệp CSV}
url=${URL:-http://api.hanni.localhost:18080/api/health/ready}
mkdir -p "$(dirname "$out")"
echo 'thoi_diem,ma' > "$out"

summary() {
  awk -F, 'NR > 1 {
    if ($2 ≠ 200) { run++; if (run > max) max = run; bad++ } else run = 0
  } END {
    printf "%d lượt, %d lượt lỗi, gián đoạn dài nhất %d giây\n", NR - 1, bad, max
  }' "$out"
  exit 0
}
trap summary INT TERM

while true; do
  code=$(curl -s -o /dev/null -m 1 -w '%{http_code}' "$url")
  echo "$(date -u +%FT%T.%3NZ),$code" >> "$out"
  sleep 1
done
```

Dòng	Đọc là
1	`#!/usr/bin/env bash` : chạy tệp này bằng bash. Dòng nào cũng có ở đầu script
2–3	Ghi chú: làm gì, gọi thế nào
4	`set -uo pipefail` : dùng biến chưa có giá trị là dừng (`-u`); lệnh nào trong chuỗi `a | b` lỗi thì cả chuỗi tính là lỗi. Script khác còn thêm `-e` : lệnh nào lỗi là dừng ngay
6	`out=${1:?Cần đường dẫn tệp CSV}` : lấy **tham số thứ nhất** (`$1`) bỏ vào biến `out` ; thiếu thì dừng và in câu báo lỗi
7	`url=${URL:-...}` : lấy biến môi trường `URL` nếu có, không thì dùng giá trị sau `:-` (giá trị mặc định)
8–9	Tạo thư mục chứa tệp kết quả; `>` ghi đè tệp bằng dòng tiêu đề CSV
11–18	Một **hàm** tên `summary` : dùng `awk` đếm lượt lỗi và chuỗi lỗi dài nhất trong tệp CSV
19	`trap summary INT TERM` : khi bạn bấm Ctrl+C thì chạy hàm `summary` trước khi thoát
21–25	Vòng lặp mãi: `curl` gọi API, `$(...)` lấy kết quả của lệnh (mã HTTP) bỏ vào biến; `>>` **ghi thêm** 1 dòng vào cuối tệp; nghỉ 1 giây

Bảng cú pháp bash hay gặp trong các script của lab:

Viết	Nghĩa
`$1` , `$2` , `"$@"`	Tham số thứ 1, thứ 2, tất cả tham số khi gọi script
`${X:-mặc định}` , `${X:?báo lỗi}`	Biến X, thiếu thì lấy mặc định / thiếu thì dừng
`$(lệnh)`	Chạy lệnh, lấy chữ nó in ra
`a | b`	Đưa kết quả lệnh a làm đầu vào lệnh b

Viết	Nghĩa
`> tệp`, `>> tệp`, `2>&1`	Ghi đè vào tệp / ghi thêm vào cuối tệp / gộp thông báo lỗi chung với kết quả
`lệnh &`, `wait`	Chạy lệnh ở nền (song song) / chờ lệnh nền xong
`case "$1" in api) … ;; esac`	Chọn nhánh theo tham số (xem `scripts/chaos.sh`)
`for x in …; do …; done`, `while …; do …; done`	Vòng lặp
`[ -f tệp ]`, `[ "$a" = "$b" ]`	Kiểm tra tệp có tồn tại / 2 chuỗi bằng nhau
`exit 1`	Thoát và báo lỗi (0 là thành công, khác 0 là lỗi)

`scripts/kubectl` cũng là 1 script, chỉ 8 dòng: nó chạy lệnh `kubectl` **bên trong** node `server` bằng `docker compose exec`, nên máy bạn không phải cài kubectl và không thể trỏ nhầm sang cụm khác. Vì vậy trong lab luôn gõ `scripts/kubectl …` thay vì `kubectl …`.

Script `.js` trong `k6/` là kịch bản tải viết bằng JavaScript (tài liệu của Vân Anh giải thích); `.py` là Python, đọc ghi chú trong cặp `"""` ở đầu tệp.

1.12 Bản đồ thư mục infra: tệp nào để làm gì

Thư mục cau-hinh-ma-nguon/hanni-server/infra/. Cột cuối là người cần nắm kỹ tệp đó.

Tệp hoặc thư mục	Để làm gì	Ai nắm
`Makefile`	Các lệnh `make up/images/deploy/seed/observability/status/stop/start/down`, mỗi lệnh là 1 nhóm lệnh viết sẵn	Cả nhóm
`compose.yaml`	Bật 2 node k3s bằng Docker: IP cố định, giới hạn RAM, mở cổng 18080	Đăng
`README.md`	Cách dựng, địa chỉ, bảng "Sửa nhanh khi bảo vệ"	Cả nhóm
`k8s/kustomization.yaml`, `k8s/namespace.yaml`	Danh sách tệp `make deploy` nạp; tạo khu `hanni`	Đăng
`k8s/app/api.yaml`	API: Deployment, Service, HPA, PDB	Đăng
`k8s/app/config.yaml`	Biến môi trường của API, có ghi chú từng dòng	Đăng
`k8s/app/worker.yaml`	2 worker: việc định kỳ, outbox	Đăng
`k8s/app/ingress.yaml`	Tên miền nào tới Service nào; giấu `/api/metrics` khỏi bên ngoài	Đăng
`k8s/app/jobs.yaml`	Việc chạy 1 lần lúc `make seed`: tạo bảng, nạp từ vựng, 1.000 tài khoản giả	Đăng
`k8s/app/media.yaml`, `k8s/media-manifest.json`	Nginx phát học liệu, kiểm chữ ký; danh sách 3 học liệu	Thành
`media/make-sample.sh`, `media/nguon/`	Dựng 3 học liệu (video HLS 2 mức, PDF, MP3) từ slide và ghi âm, ghi mã băm	Thành

Tệp hoặc thư mục	Để làm gì	Ai nắm
`k8s/data/postgres.yaml`, `postgres-replica.yaml`	CSDL chính (bật lưu nhật ký WAL) và bản sao đọc	Đăng
`k8s/data/pgbouncer.yaml`, `redis.yaml`	Gom kết nối CSDL; Redis cho giới hạn request, bộ đệm, khoá	Đăng
`k8s/policy/network-policy.yaml`	Chặn mạng trong cụm, chỉ mở đúng đường đi	Đức
`k8s/backup/cronjob.yaml`, `backup.sh`	Lịch và script sao lưu hằng ngày	Đức
`k6/lib.js`	Phần dùng chung của kịch bản tải, có tiêu chí đạt	Vân Anh
`k6/kb1-tu-hoc.js`, `kb2-video.js`, `kb3-thi-thu.js`	3 kịch bản: tự học, cả lớp xem video, thi thử đồng loạt	Vân Anh
`k6/tang-dan.js`, `dang-nhap.js`, `kb1-dong.js`, `ghi-lien-tuc.js`	Tìm điểm gãy; đăng nhập đồng loạt; mô hình đóng; ghi liên tục khi diễn tập	Vân Anh
`k6/thu-tai-nguyen.sh`, `scripts/k6.sh`	Ghi CPU/RAM từng pod khi đo; chạy k6 trong Docker	Vân Anh
`scripts/chay-kt2.sh`, `cho-du-ban.sh`, `chon-muc.py`	Kiểm thử 2; chờ đủ bản API rồi mới đo; chọn 3 mức từ đo thăm dò	Vân Anh
`scripts/kt1.sh`	Kiểm thử 1: tải đủ học liệu như máy khách, so mã băm	Vân Anh, Thành
`scripts/kt3-dung-dich-vu.sh`, `kt3-mat-du-lieu.sh`, `do-gian-doan.sh`	Kiểm thử 3; đo thời gian gián đoạn	Đức
`scripts/chaos.sh`	Diễn tập: tắt 1 bản API, Redis, Postgres, 1 node	Đức
`scripts/restore.sh`, `chep-sao-luu.sh`, `loi-sao-luu.sh`, `so-sanh-sao-luu.sh`	Khôi phục; chép bản sao lưu ra ngoài cụm; thử lỗi sao lưu; so pg_dump với pg_basebackup	Đức
`scripts/kiem-quyen.sh`	Kiểm thử phân quyền	Đức
`scripts/failover.sh`, `doi-chieu-ghi.sh`, `pitr.sh`, `do-phan-manh.sh`, `sql/`	Chuyển sang bản sao; đếm lượt ghi mất; khôi phục về đúng thời điểm; chia bảng	Đăng
`scripts/kubectl`, `secrets.sh`, `cho-du-node.sh`, `xuat-chi-so.py`	kubectl của lab; sinh mật khẩu; chờ đủ 2 node; xuất số Prometheus ra CSV	Đăng

Tệp hoặc thư mục	Để làm gì	Ai nắm
`scripts/so-sanh-video.cjs`, `quay-demo.cjs`, `ci/quay-demo.sh`	So HLS với MP4 trên trình duyệt; quay video demo	Thành
`observability/`	Prometheus, Alertmanager, Grafana (3 bảng ở `dashboards/`), Loki, Alloy, Jaeger, blackbox	Đăng
`observability/rules/hanni-slo.yml`	Luật cảnh báo: khi nào Alertmanager báo	Đức
`otel/tracing.js`	Gắn theo dõi đường đi request để xem trên Jaeger	Đăng
`ci/chung.sh`, `kt2.sh`, `kt3.sh`, `phu.sh`	Các job đo trên GitHub Actions (workflow `.github/workflows/do-lab.yml`)	Đăng

1.13 Xem cụm đang chạy: các lệnh kubectl cơ bản

Chạy trong thư mục `hanni-server/infra`. Thêm `-n hanni` để xem khu ứng dụng, `-n observability` để xem khu giám sát.

Lệnh	Cho biết gì
`scripts/kubectl get nodes`	2 node, trạng thái `Ready`
`scripts/kubectl -n hanni get pods -o wide`	Mọi pod, trạng thái, nằm trên node nào
`scripts/kubectl -n hanni get deploy,statefulset,svc`	Các Deployment, StatefulSet, Service
`scripts/kubectl -n hanni get hpa`	Số bản API hiện tại, CPU hiện tại so với mục tiêu 65%
`scripts/kubectl -n hanni describe pod <tên>`	Mọi chi tiết của 1 pod; mục **Events** cuối cùng cho biết vì sao pod lỗi
`scripts/kubectl -n hanni logs <tên pod>`	Nhật ký của chương trình trong pod (thêm `-f` để xem liên tục)
`scripts/kubectl -n hanni get events --sort-by=.lastTimestamp`	Mọi sự kiện gần đây: tạo pod, thêm bớt bản, lỗi
`scripts/kubectl top pods -n hanni`	CPU (mili lõi, `250m` = 0,25 lõi), RAM từng pod lúc này
`scripts/kubectl -n hanni rollout restart deployment/hanni-api`	Khởi động lại API lần lượt từng bản, không gián đoạn
`scripts/kubectl -n hanni scale deployment hanni-api --replicas=0`	Đổi số bản bằng tay (0 là dừng hẳn, dùng trong Kiểm thử 3)

Ví dụ đầu ra thật của `get pods -o wide` cuối lượt đo 09/11 (đã bớt cột, tệp gốc `data/raw/gha-09-11/ket-qua-kt3/pods-cuoi.txt`):

```
NAME                                READY  STATUS     RESTARTS  AGE    NODE
hanni-api-5456c9c9cb-5xzh5          1/1    Running    0         6m8s   k3s-agent
hanni-api-5456c9c9cb-b9sw4          1/1    Running    0         6m8s   k3s-server
hanni-migrate-zbbm6                 0/1    Completed  0         32m    k3s-agent
hanni-postgres-replica-0            2/2    Running    0         32m    k3s-server
hanni-worker-7fbffb6b7c-fcm85       1/1    Running    0         6m8s   k3s-server
hanni-worker-7fbffb6b7c-gxvkh       1/1    Running    0         6m8s   k3s-agent
```

- `READY 1/1`: pod có 1 container, đã sẵn sàng. `2/2`: pod có 2 container, ví dụ bản sao Postgres chạy kèm 1 container đưa số liệu cho Prometheus.
- `STATUS Running`: đang chạy. `Completed`: Job đã làm xong (ở đây `hanni-migrate` tạo bảng lúc dựng lab). Gặp `Pending`, `CrashLoopBackOff`, `ErrImageNeverPull` thì xem mục 1.8.
- `RESTARTS`: số lần bị khởi động lại; tăng dần là có vấn đề, xem `logs`.
- `NODE`: pod nằm ở node nào; 2 bản API ở 2 node khác nhau nhờ dòng 28–34 của `api.yaml`.
- Không có `hanni-postgres-0` vì tệp này chụp **sau** diễn tập chuyển sang bản sao: CSDL chính cũ đã bị tắt, `hanni-postgres-replica-0` đang làm CSDL chính.

1.14 Buổi thực hành đầu tiên (khoảng 1 giờ)

Làm hết buổi này một lần là bạn đã tự tay: dựng lab, xem cụm, xem Grafana, tạo tải, thấy tự mở rộng, sửa 1 cấu hình rồi hoàn tác, gây 1 sự cố và thấy hệ thống tự hồi phục. Mọi lệnh chạy trong `~/hanni-server/infra`.

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
1. Dựng lab	Theo mục 1.6	`make status`: mọi pod `Running` hoặc `Completed`	Mục 1.8
2. Xem 2 node	`scripts/kubectl get nodes`	2 dòng `k3s-server`, `k3s-agent` đều `Ready`	`make start`, chờ 1 phút
3. Xem các bản API	`scripts/kubectl -n hanni get pods -l app=hanni-api -o wide`	2 pod `Running`, cột NODE mỗi pod 1 node	Chờ 1 phút, gõ lại
4. Xem tự mở rộng	`scripts/kubectl -n hanni get hpa`	`MINPODS 2`, `MAXPODS 6`, `REPLICAS 2`, `TARGETS` dạng `cpu: 4%/65%`	`<unknown>` thì chờ 1 phút
5. Lấy mật khẩu Grafana	`scripts/kubectl -n observability get secret grafana-admin -o jsonpath='{.data.password}' | base64 -d; echo`	1 chuỗi ký tự ngẫu nhiên	Chưa `make observability` thì chạy
6. Mở Grafana	Trình duyệt: http://grafana.hanni.localhost:18080, đăng nhập `admin` + mật khẩu bước 5, vào http://grafana.hanni.localhost:18080/d/hanni-scaling	Bảng "Hanni: Mở rộng", ô "Số bản API" = 2	Mục 1.15

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
7. Tạo tải	Mở tab terminal thứ 2: `scripts/k6.sh kb1-tu-hoc.js -e RATE=20 -e DURATION=3m`	Trên Grafana sau 1–2 phút: CPU tăng, "Số bản API" có thể lên 3–4; k6 chạy xong in bảng tóm tắt (xem dưới)	Máy yếu thì giảm `RATE=8`
8. Sửa 1 cấu hình	VS Code mở `infra/k8s/app/api.yaml`, Ctrl+G dòng 143, đổi `maxReplicas: 6` thành `4`, Ctrl+S; rồi `make deploy`	`scripts/kubectl -n hanni get hpa` hiện `MAXPODS 4`	Mục 0.6, sửa đúng thụt lề
9. Xem mình đã sửa gì	`git diff`	1 dòng `- maxReplicas: 6` và 1 dòng `+ maxReplicas: 4`	
10. Hoàn tác	`git restore k8s/app/api.yaml` rồi `make deploy`	`get hpa` hiện lại `MAXPODS 6`, `git status` không còn tệp đỏ	
11. Gây sự cố	`scripts/chaos.sh api` (xoá đột ngột 1 bản API)	Gõ ngay lệnh bước 3: 1 pod mới có AGE vài giây, vẫn đủ 2 bản	
12. Tắt lab	`make stop`	Dấu nhắc trở lại; `docker ps` không còn 2 container	

Bảng tóm tắt k6 in ra cuối bước 7 có dạng như sau (đây là dòng thật của 1 lần đo chính thức, mức 36 lượt/giây; bước 7 của bạn tải nhẹ hơn nên số khác):

```
     ✓ lộ trình 200
     checks.....................: 100.00% 54005 out of 54005
     ✓ { kind:read }.............: avg=15.28ms  min=2.81ms  med=8.96ms   max=302.96ms
p(90)=30.74ms  p(95)=41.47ms  p(99)=79.8ms
   ✓ http_req_failed.............: 0.00%   0 out of 54006
     iterations.....................: 10801   35.887685/s
```

Đọc nhanh: dấu `✓` là đạt, `✗` là không đạt; `checks 100%` là mọi request trả đúng mã; `p(95)=41.47ms` là 95% lượt đọc nhanh hơn 41 ms (ngưỡng đạt là 300 ms); `http_req_failed 0.00%` là không lỗi; `iterations` là số lượt đã chạy và số lượt mỗi giây. Chi tiết ở tài liệu của Vân Anh.

> Bạn vừa tự tay làm đúng những việc thầy có thể yêu cầu khi bảo vệ: mở minh chứng (Grafana), đổi 1 thông số rồi giải thích (bước 8–10), cho thấy hệ thống tự hồi phục (bước 11). Làm lại buổi này 2 lần là đủ tự tin.

1.15 Đọc Grafana, Prometheus, Alertmanager

Ba trang web của phần giám sát, đều mở qua cổng 18080 (lab phải đang chạy và đã `make observability`):

Trang	Địa chỉ	Dùng để
Grafana	http://grafana.hanni.localhost:18080	Xem biểu đồ. Tài khoản `admin`, mật khẩu lấy ở bước 5 mục 1.14
Prometheus	http://prometheus.hanni.localhost:18080	Nơi lưu số đo. Menu Status → Targets: mọi nguồn số liệu phải `UP`
Alertmanager	http://alertmanager.hanni.localhost:18080	Danh sách cảnh báo **đang bật**. Trống là không có sự cố

Grafana (giao diện tiếng Anh): menu trái **Dashboards** liệt kê 3 bảng của nhóm, hoặc gõ thẳng địa chỉ. Góc trên phải có ô chọn khoảng thời gian (mặc định vài phút gần nhất) và nút làm mới. Rê chuột lên biểu đồ để xem số tại từng thời điểm.

Bảng	Địa chỉ	Các ô quan trọng
Hanni: SLO và tín hiệu vàng	`/d/hanni-slo`	Độ sẵn sàng 1 giờ (mục tiêu 99,5%), Đọc p95 (dưới 300 ms), Ghi p95 (dưới 500 ms), Lưu lượng, Tỉ lệ lỗi 5 phút, Cảnh báo đang bật, Dò từ ngoài qua ingress (1 là vào được, vẫn đo được khi mọi bản API đã chết), 10 route chậm nhất
Hanni: Mở rộng	`/d/hanni-scaling`	Số bản API, Trần HPA, Request chờ kết nối CSDL, Event loop trễ, Kết nối Postgres (trần 60), CPU từng bản API so với yêu cầu 250 mCPU (vượt 65% là HPA thêm bản), CPU bị bóp do chạm giới hạn, RAM từng pod
Hanni: Dữ liệu	`/d/hanni-du-lieu`	Postgres, Redis có sống không, Dung lượng CSDL, Sao lưu gần nhất (cảnh báo khi quá 26 giờ), Bản sao trễ, Outbox chưa xử lý, Giao dịch mỗi giây

Cảnh báo (luật ở `observability/rules/hanni-slo.yml`): mỗi luật có điều kiện và thời gian phải kéo dài (`for`) mới báo, để không báo nhầm vì 1 giây chập chờn.

Tên cảnh báo	Báo khi
HanniEdgeDown	Dò từ ngoài không gọi được API, kéo dài 30 giây (cảnh báo dùng trong Kiểm thử 3)
HanniApiDown	Không còn bản API nào trả lời, 1 phút
HanniPostgresDown	Không kết nối được PostgreSQL, 1 phút
HanniReadLatencyHigh, HanniWriteLatencyHigh	Đọc p95 vượt 300 ms / ghi p95 vượt 500 ms, 5 phút
HanniErrorBudgetBurnFast, …Slow	Tỉ lệ lỗi 5xx cao tới mức giữ nhịp này thì hết "ngân sách lỗi" của tháng trong 2 ngày / 5 ngày
HanniHpaAtMax	API đã chạm trần 6 bản, 5 phút: tự mở rộng hết sức
HanniDbPoolWaiting	Có request xếp hàng chờ kết nối CSDL, 2 phút
HanniEventLoopLag	Node.js trễ quá 200 ms, 5 phút: API nghẽn CPU
HanniReplicationLagHigh	Bản sao đọc trễ quá 30 giây
HanniOutboxBacklog	Hơn 500 việc phụ sau nộp bài chưa xử lý, 5 phút
HanniBackupStale	Hơn 26 giờ chưa có bản sao lưu thành công

Lab chưa gắn kênh gửi thật (email, Telegram), nên cảnh báo chỉ hiện trên Alertmanager và ô "Cảnh báo đang bật" của Grafana.

Phần 2. Phần việc của bạn

2.1 Phiếu đề giao gì cho bạn

Phiếu đề ghi Sinh viên 3 lo **"Thiết lập tải và thu thập số liệu"**: tạo tải giả, đo, ghi số liệu sao cho người khác chạy lại được.

Vai trò	Bạn làm gì	Phục vụ phần nào của đề
QA Lead	Kế hoạch kiểm thử, tiêu chí đạt, bảng kiểm thử có giờ chạy	Kiểm thử 1, hồ sơ nộp
Performance Test Engineer	Kịch bản k6, đo thăm dò, Kiểm thử 2 (3 mức × 3 lần), đo đăng nhập, nộp bài, tự mở rộng	Kiểm thử 2, Yêu cầu 4
Phân tích số liệu	Tổng hợp p50/p95/p99, thông lượng, lỗi, CPU; chỉ ra điểm nghẽn	Yêu cầu 4

2.2 Sản phẩm của bạn trong hồ sơ

Sản phẩm	Ở đâu
Kịch bản k6	infra/k6/
Script chạy Kiểm thử 2, chờ đủ bản, thu CPU/RAM	scripts/chay-kt2.sh, scripts/cho-du-ban.sh, k6/thu-tai-nguyen.sh
Dữ liệu gốc Kiểm thử 2	data/raw/gha-09-11/ket-qua-kt2/
Bảng tổng hợp	data/kt2-tong-hop.xlsx
Bảng kiểm thử	bang-kiem-thu.xlsx
Đăng nhập, nộp bài, tự mở rộng, 2 cách tạo tải	data/raw/gha-09-11/ket-qua-phu/
Báo cáo cá nhân	RIPT1304-20261-01_NhomXX_06_B24DCCC007_BaoCaoCaNhan.docx

Phần 3. Kiểm thử hiệu năng và k6 từ đầu

3.1 Kiểm thử hiệu năng là gì, khác kiểm thử chức năng thế nào

- **Kiểm thử chức năng** hỏi "bấm nút này có ra đúng kết quả không?". Một người bấm là đủ.
- **Kiểm thử hiệu năng** hỏi "**khi nhiều người cùng bấm**, hệ thống trả lời nhanh không, có lỗi không, chịu được tới mức nào?". Không thể nhờ 1.000 người thật bấm, nên dùng công cụ **giả lập** nhiều người: k6.

3.2 k6 hoạt động thế nào

k6 là chương trình chạy một **kịch bản** (tệp JavaScript) mô tả 1 sinh viên làm gì, rồi chạy kịch bản đó rất nhiều lần cùng lúc.

- **VU (người dùng ảo)**: 1 "sinh viên giả" đang chạy kịch bản. k6 có thể chạy hàng trăm VU.
- **Iteration (lượt)**: 1 lần chạy trọn kịch bản. Trong `kb1-tu-hoc.js`, 1 lượt = mở lộ trình, xem chuỗi ngày, mở phiên học, tra từ, xem từ hôm nay = **5 request đọc**.
- **Executor (cách phát tải)**: nhóm dùng `constant-arrival-rate`: **mỗi giây bắt đầu đúng RATE lượt mới**, bất kể lượt cũ đã xong chưa.
- **Tag** `kind: read` / `kind: write`: đánh dấu request đọc hay ghi để tính riêng p95 đọc (ngưỡng 300 ms) và p95 ghi (ngưỡng 500 ms).
- **Threshold (ngưỡng đạt)**: k6 tự so kết quả với ngưỡng; không đạt thì thoát với mã lỗi, script thu số liệu biết lần đó trượt.
- **dropped_iterations**: số lượt k6 **định phát mà không phát được** (thiếu VU). Khác 0 nghĩa là tải thật thấp hơn tải đã khai, số đo không dùng được.

Kịch bản tự học, phần mỗi lượt làm gì:

hanni-server/infra/k6/kb1-tu-hoc.js, dòng 30–39 (mở trên GitHub):

```
  30 | export default function (data) {
  31 |   const s = session();
  32 |   const read = { ...s, tags: { kind: 'read' } };
  33 |   ok(get('/learn/path?level=1', read), 'lộ trình');
  34 |   ok(get('/streak', read), 'chuỗi ngày');
  35 |   ok(get(`/learn/lessons/${data.lessonId}/session`, read, '/learn/lessons/:id/session'),
'phiên học');
  36 |   ok(get('/words?q=hoc&pageSize=10', read, '/words?q='), 'tra từ');
  37 |   sleep(Math.random());
  38 |   ok(get('/words/of-the-day?level=1', read), 'từ hôm nay');
  39 | }
```

Cấu hình cách phát tải:

hanni-server/infra/k6/kb1-tu-hoc.js, dòng 10–19 (mở trên GitHub):

```
  10 |   scenarios: {
  11 |     tu_hoc: {
  12 |       executor: 'constant-arrival-rate',
  13 |       rate: RATE,
  14 |       timeUnit: '1s',
  15 |       duration: __ENV.DURATION || '5m',
  16 |       preAllocatedVUs: RATE * 5,
  17 |       // Có trần để server chậm thì k6 không ăn hết RAM máy đo; chạm trần sẽ thấy ở
dropped_iterations.
  18 |       maxVUs: Number(__ENV.MAX_VUS || RATE * 40),
  19 |       gracefulStop: '30s',
```

- `rate: RATE`, `timeUnit` 1 giây: mỗi giây bắt đầu RATE lượt.
- `preAllocatedVUs: RATE * 5`: chuẩn bị sẵn VU; `maxVUs`: tối đa bao nhiêu VU khi máy chủ chậm (lượt cũ chưa xong thì cần thêm VU để vẫn giữ nhịp).

Ngưỡng đạt dùng chung cho mọi kịch bản:

hanni-server/infra/k6/lib.js, dòng 40–45 (mở trên GitHub):

```
  40 | /** Số trong SLO. Vượt thì k6 thoát mã khác 0 để script thu số liệu biết lần đo đó không đạt.
*/
  41 | export const SLO_THRESHOLDS = {
  42 |   'http_req_duration{kind:read}': ['p(95)<300'],
  43 |   'http_req_duration{kind:write}': ['p(95)<500'],
  44 |   http_req_failed: ['rate<0.005'],
  45 | };
```

Đọc: 95% lượt đọc dưới 300 ms, 95% lượt ghi dưới 500 ms, tỉ lệ lỗi dưới 0,5%. Tệp `lib.js` còn **chặn đo nhầm web thật**: chỉ nhận tên miền `.localhost` hoặc IP nội bộ.

3.3 Các kịch bản của nhóm

Kịch bản	Giả lập	Tham số
kb1-tu-hoc.js	Sinh viên tự học: 5 request đọc mỗi lượt	`RATE` lượt/giây, `DURATION`
kb2-video.js	Cả lớp xem video: mỗi người tải 1 đoạn 6 giây	`VIEWERS` người xem
kb3-thi-thu.js	Thi thử: mở đề trong 60 giây, nộp trong 30 giây	`USERS` người
tang-dan.js	Tăng tải dần tới điểm gãy	`MAX_RATE`, `STEP`, `HOLD`
dang-nhap.js	Đăng nhập đồng loạt, mỗi lượt 1 tài khoản	`RATE`, `PASS`
kb1-dong.js	Tự học theo mô hình đóng (để so sánh)	`VUS`

3.4 Mô hình mở và mô hình đóng (câu hỏi rất hay bị hỏi)

- **Mô hình mở** (nhóm dùng): mỗi giây có thêm RATE người mới vào, **dù máy chủ đang chậm**. Giống đời thật: sinh viên không đợi người khác học xong mới vào. Máy chủ chậm thì hàng đợi dài ra và p95 tăng, ta **thấy được** vấn đề.
- **Mô hình đóng**: có N người ảo cố định, mỗi người **chờ trả lời xong** mới gửi tiếp. Máy chủ chậm thì họ tự gửi chậm lại, tải giảm theo, số đo trông đẹp hơn thực tế (gọi là **coordinated omission**, "bỏ sót do phối hợp").

Nhóm đã đo cả 2 trên cùng lab (mục 4.4) để chứng minh bằng số.

3.5 Phân vị p50, p95, p99 và độ lệch

Ví dụ 100 request có thời gian trả lời xếp từ nhanh tới chậm. **p50** là request thứ 50 (một nửa nhanh hơn), **p95** là request thứ 95, **p99** là request thứ 99. Trung bình dễ bị đánh lừa: 95 request 10 ms và 5 request 2.000 ms cho trung bình 109 ms, nghe ổn, nhưng 5% người dùng phải chờ 2 giây. p95 lộ ra điều đó.

Độ lệch (±): mỗi mức đo 3 lần, độ lệch cho biết 3 lần chênh nhau bao nhiêu. Độ lệch nhỏ là kết quả ổn định, tin được.

Tính độ lệch bằng tay (độ lệch chuẩn mẫu, hàm `STDEV.S` trong Excel, `statistics.stdev` trong Python), ví dụ thật: p95 đọc của 3 lần đo mức 36 lượt/giây, bản sau cải tiến, lấy từ 3 tệp `muc-36-lan-1/2/3.json`:

Bước	Cách làm	Kết quả
1. 3 số đo	Đọc dòng `http_req_duration{kind:read}`, cột p(95)	41,48 ; 37,11 ; 40,47 ms
2. Trung bình	Cộng 3 số rồi chia 3	39,68 ms
3. Chênh lệch từng lần	Lấy từng số trừ trung bình	1,79 ; -2,58 ; 0,78
4. Bình phương rồi cộng	Bình phương từng chênh lệch, cộng lại	10,46
5. Chia cho (số lần - 1)	Chia cho 2 (vì đo 3 lần)	5,23
6. Căn bậc hai	Lấy căn của số trên	**2,29 ms**

Vậy ghi là **40 ± 2 ms**, khớp dòng mức 36 "Sau" trong data/kt2-tong-hop.xlsx. Chia cho (số lần − 1) chứ không chia cho số lần vì 3 lần chỉ là **mẫu** của vô số lần có thể đo; chia cho n − 1 bù cho việc mẫu nhỏ thường lệch ít hơn thực tế. Cách đọc nhanh: độ lệch nhỏ hơn khoảng 10% trung bình là ổn định. Bản trước cải tiến ở cùng mức có độ lệch 16 ms trên trung bình 57 ms: 3 lần chênh nhau nhiều, dấu hiệu hệ thống đang sát giới hạn.

3.6 Thông lượng, điểm gãy, điểm nghẽn

- **Thông lượng** (request/giây): số request trả lời được mỗi giây. Dưới điểm gãy, thông lượng bằng tải đưa vào.
- **Điểm gãy**: mức tải cao nhất còn đạt ngưỡng. Quá đó p95 tăng vọt, có khi thông lượng còn **giảm** vì request xếp hàng giữ tài nguyên lâu hơn.
- **Điểm nghẽn**: bộ phận bị đầy đầu tiên. Bạn tìm nó bằng Grafana: lúc p95 tăng vọt, xem CPU API, hàng chờ kết nối CSDL, bộ phận nào chạm trần trước.

Phần 4. Kiểm thử 2: đo, đọc kết quả, tổng hợp

4.1 Bước 1: đo thăm dò để chọn 3 mức

Không thể đoán 3 mức tải "phù hợp thiết bị" mà đề yêu cầu, nên đo thử tăng dần 10 → 80 lượt/giây, mỗi mức 1 phút:

Lượt/giây	Request/giây	p95 đọc (ms)	Tỉ lệ lỗi	Đạt
10	49,5	23,2	0,00%	Có
20	98,9	23,9	0,00%	Có
30	148,1	30,9	0,00%	Có
40	197,0	52,1	0,00%	Có
50	244,7	261,4	1,09%	Không
60	292,5	688,0	24,85%	Không
70	341,8	767,8	37,79%	Không
80	386,5	824,7	46,53%	Không

Mức cao nhất còn đạt là **40**. 3 mức lấy 25%, 60%, 90% của 40: **10, 24, 36 lượt/giây** (khoảng 50, 120, 180 request/giây). Vì sao không lấy sát 40: để cả 3 mức đều đo được ổn định, không lần nào rơi đúng mép gãy. Lỗi từ 50 lượt trở lên đều là **503 chủ động** (cơ chế từ chối khi quá tải), không có lỗi máy chủ thật.

4.2 Bước 2: chạy Kiểm thử 2 cho đúng cách

Script chay-kt2.sh làm tự động, mỗi lần đo:

1. **Cố định 6 bản API** (tắt tự mở rộng trong lúc đo) để các lần so được với nhau.
2. Chạy cho-du-ban.sh: chờ đủ 6 bản sẵn sàng và không còn bản nào đang tắt, thêm 20 giây cho cổng vào cập nhật.
3. **Làm nóng 1 phút** (không tính): API vừa khởi động còn chậm, đo ngay là đo nhầm.
4. **Đo 5 phút**, đồng thời thu-tai-nguyen.sh ghi CPU, RAM từng pod mỗi 5 giây.
5. Ghi số trang swap: lần nào máy phải dùng swap thì bỏ, đo lại.
6. Ghi **điều kiện đo** vào `dieu-kien.txt` để người khác lặp lại được.

Lệnh: `scripts/chay-kt2.sh k6/ket-qua/kt2 "10 24 36" 3` (3 mức, mỗi mức 3 lần). Tệp điều kiện đo thật của lượt chính thức:

data/raw/gha-09-11/ket-qua-kt2/kt2-sau/dieu-kien.txt (mở trên GitHub):

```
Thời điểm: 2026-11-09 12:02:13 +0000
Commit: f4e29d4
Image API: hanni-api:lab
Số bản API: 6, mỗi bản tối đa 1 CPU, 512 MB
Máy: AMD EPYC 9V74 80-Core Processor, 4 luồng, 15983 MB RAM, Ubuntu 24.04.5 LTS
k6 chạy chung máy với hệ thống, đi qua loopback tới ingress Traefik
Làm nóng 1m, đo 5m, 3 lần mỗi mức, mức (lượt mở bài/giây): 10 24 36
"CRON_LEASE":"true"
```

4.3 Đọc kết quả k6 (tệp thật, mức 36 lượt/giây, lần 1)

data/raw/gha-09-11/ket-qua-kt2/kt2-sau/muc-36-lan-1.txt (mở trên GitHub):

```
          * tu_hoc: 36.00 iterations/s for 5m0s (maxVUs: 180-400, gracefulStop: 30s)
     http_req_duration..............: avg=15.28ms  min=2.81ms  med=8.96ms   max=302.96ms
p(90)=30.74ms  p(95)=41.47ms  p(99)=79.8ms
   ✓ http_req_failed................: 0.00%   0 out of 54006
     http_reqs......................: 54006   179.441747/s
     iterations.....................: 10801   35.887685/s
     vus............................: 1       min=1              max=33
     vus_max........................: 180     min=180            max=180
```

Dòng	Đọc là
`tu_hoc: 36.00 iterations/s for 5m0s`	Đã phát đúng 36 lượt mỗi giây suốt 5 phút
`http_req_duration … p(95)=41.47ms`	95% request trả lời dưới 41 ms, xa ngưỡng 300 ms
`max=302.96ms`	Request chậm nhất 303 ms (chỉ 1 vài cái, không ảnh hưởng p95)
`http_req_failed: 0.00%`	Không request nào lỗi
`http_reqs: 54006 179.44/s`	Tổng 54.006 request, trung bình 179 request/giây (36 lượt × 5 request)
`vus … max=33`, `vus_max 180`	Cần tối đa 33 VU đồng thời, chuẩn bị sẵn 180 (đủ, không bị dropped)

Tệp `-tai-nguyen.csv` cùng tên ghi CPU (mili lõi), RAM (MiB) từng pod mỗi 5 giây; cột `so_ban_api` xác nhận 6 bản suốt lần đo.

4.4 Bước 3: tổng hợp

data/kt2-tong-hop.xlsx lấy trung bình và độ lệch 3 lần mỗi mức:

Mức	Bản	Request/giây	p50	p95 ± độ lệch	p99	Lỗi	CPU API (lõi)
10	Trước	49,9	7	22 ± 0	28	0,00%	0,49
10	Sau	49,9	6	21 ± 0	28	0,00%	0,46
24	Trước	119,6	7	26 ± 1	42	0,00%	1,11
24	Sau	119,7	7	25 ± 1	40	0,00%	1,06
36	Trước	179,5	12	57 ± 16	104	0,00%	1,81
36	Sau	179,5	9	40 ± 2	75	0,06%	1,65

Đọc bảng: cả 3 mức đều đạt ở cả 2 bản. Ở 2 mức thấp, 2 bản gần như bằng nhau vì chưa chạm giới hạn nào. Ở mức cao, bản sau nhanh hơn và **ổn định hơn nhiều** (độ lệch 16 → 2 ms). "Trước" là bản API chưa sửa điểm nghẽn, đo cùng máy, cùng lượt.

4.5 Đo thêm

Phép đo	Cách làm	Kết quả, ý nghĩa
Đăng nhập đồng loạt	`dang-nhap.js`, mỗi lượt 1 tài khoản khác, 10/20/40 lượt/giây, 3 lần	10/giây: p95 55 ms; 20/giây: p95 58 ms; 40/giây: p95 72 ms; 0 lỗi. 1.000 sinh viên đăng nhập trong 1 phút chỉ cần khoảng 17 lượt/giây, nên đăng nhập không phải điểm nghẽn
Nộp bài đồng loạt	`kb3-thi-thu.js`, 600 người, 10% gửi lại 2 lần	Số bài ghi vào CSDL đúng bằng số lượt gửi (601, 600, 601); gửi lại không thành 2 bài
Tự mở rộng	`tang-dan.js` tới 200 request/giây, xem số bản	4 → 3 → 5 → 6 bản
Mô hình mở và đóng	Cùng lab, cùng kịch bản, 3 lần mỗi mức	Bảng dưới

Mức	Mở: p95	Đóng: p95
15 lượt/giây	27 ms	38 ms
30 lượt/giây	40 ms	61 ms

Kết quả **ngược với dự đoán**: khi chưa quá tải, mô hình đóng còn cho p95 cao hơn, vì người ảo gửi dồn ngay khi lượt trước xong. Mô hình đóng chỉ "che" độ chậm khi máy chủ đã quá tải. Nhóm vẫn dùng mô hình mở. Ghi đúng như đo là điểm cộng, không phải điểm trừ.

4.6 Vấn đề đã gặp: lỗi của phép đo, không phải lỗi hệ thống

Lần chạy thử đầu có 2,2% lỗi ở mức thấp nhất, mức cao hơn lại 0 lỗi, rất vô lý. Đối chiếu với Prometheus thì phía API **không ghi lỗi nào**. Lỗi rơi đúng lúc Kubernetes đang tăng số bản API từ 3 lên 6: cổng vào chuyển request tới bản chưa sẵn sàng. Cách sửa: thêm `cho-du-ban.sh` chờ đủ bản rồi mới đo. Bài học: **phải đối chiếu số k6 với số phía máy chủ trước khi kết luận hệ thống lỗi**.

4.7 Đọc bảng kiểm thử

bang-kiem-thu.xlsx là hồ sơ đề bắt buộc: **mỗi dòng là 1 lần chạy có thật**, 62 dòng, đều đạt. Các cột:

Cột	Ý nghĩa	Ví dụ
Ca kiểm thử	Tên ca bằng lời thường, kèm mức tải, số lần	"Kiểm thử 2: mức 36 lượt/giây, lần 1, sau cải tiến"
Phần của đề	Ca này chứng minh cho phần nào của phiếu đề	Kiểm thử 2, Yêu cầu 4
Đầu vào	Điều kiện trước khi chạy	6 bản API, làm nóng 1 phút
Thao tác	Lệnh hoặc script đã chạy	`scripts/chay-kt2.sh`, đo 5 phút
Kết quả mong đợi	Tiêu chí đạt, đặt **trước** khi chạy	p95 đọc dưới 300 ms, lỗi dưới 0,5%
Kết quả thực tế	Số đo thật lấy từ tệp gốc	request/giây, p95, p99, tỉ lệ lỗi

Cột	Ý nghĩa	Ví dụ
Kết luận	Đạt / Chưa đạt / Chưa đủ bằng chứng	Đạt
Thời điểm chạy	Giờ chạy ghi trong chính tệp gốc (giờ UTC, giờ Việt Nam = UTC + 7)	09/11/2026 12:39:28 UTC
Phụ trách, nơi chạy	Người chịu trách nhiệm ca đó (chạy lại, giải thích khi bảo vệ) và máy đã chạy	Vân Anh · máy ảo GitHub, 09/11
Bằng chứng	Đường dẫn tệp gốc trong `ho-so-nop/`	`data/raw/gha-09-11/ket-qua-kt2/...`

Cách kiểm 1 dòng trước mặt thầy: mở tệp ở cột Bằng chứng, tìm đúng số ở cột Kết quả thực tế (ví dụ dòng `http_req_duration{kind:read}`), đọc giờ chạy ở đầu tệp. Các dòng ghi "lượt GitHub Actions 42611229757" là ca mà tệp gốc không tự ghi giờ: giờ từng bước nằm trong log của lượt chạy đó trên GitHub. Bảng được dựng tự động từ tệp gốc nên không có số gõ tay; nếu chạy lại ca nào thì thêm dòng mới, không sửa dòng cũ.

Phần 5. Sửa và cấu hình trực tiếp

5.1 Đổi mức tải, thời gian đo

Không phải sửa tệp, chỉ đổi tham số:

```
scripts/k6.sh kb1-tu-hoc.js -e RATE=8 -e DURATION=1m      # 8 lượt/giây trong 1 phút
scripts/k6.sh kb2-video.js -e VIEWERS=60                  # 60 người xem video
scripts/k6.sh kb3-thi-thu.js -e USERS=300                 # 300 người thi thử
scripts/k6.sh tang-dan.js -e MAX_RATE=40 -e STEP=25s      # tăng dần tới 40 lượt/giây, mỗi bậc 25
giây
```

Kết quả tóm tắt in ra màn hình và lưu ở `infra/k6/ket-qua/`. Đọc dòng `http_req_duration` (p95) và `http_req_failed` (tỉ lệ lỗi).

5.2 Đổi 3 mức của Kiểm thử 2, số lần mỗi mức

`scripts/chay-kt2.sh k6/ket-qua/kt2 "10 24 36" 3` → đổi chuỗi trong ngoặc kép (3 mức) và số cuối (số lần). Mỗi lần đo tạo `muc-<mức>-lan-<lần>.txt`, `.json`, `-tai-nguyen.csv` trong thư mục kết quả, kèm `dieu-kien.txt`.

5.3 Đổi tiêu chí đạt

hanni-server/infra/k6/lib.js, dòng 41–45 (mở trên GitHub):

```
41 | export const SLO_THRESHOLDS = {
42 |   'http_req_duration{kind:read}': ['p(95)<300'],
43 |   'http_req_duration{kind:write}': ['p(95)<500'],
44 |   http_req_failed: ['rate<0.005'],
45 | };
```

Ví dụ muốn đọc phải dưới 200 ms: đổi `'p(95)<300'` thành `'p(95)<200'`. Chạy lại một kịch bản: nếu không đạt, k6 in dấu ✗ cạnh dòng đó và thoát với mã khác 0.

5.4 Cho thầy xem 1 lần đo bất kỳ

Mở data/raw/gha-09-11/ket-qua-kt2/kt2-sau/, chọn `muc-24-lan-2.txt`, chỉ dòng p95, rồi mở kt2-tong-hop.xlsx chỉ ra số đó nằm ở đâu trong bảng tổng hợp (mức 24, lần 2).

Phần 6. Chạy lại và số liệu của bạn

6.1 Chạy lại phần của bạn, từng bước

Mọi lệnh chạy trong `~/hanni-server/infra` khi lab đang chạy (mục 1.6). Cột "Thấy gì là đúng" lấy từ đầu ra thật của lượt đo chính thức 09/11, số của bạn có thể khác chút ít; thấy khác hẳn thì xem cột cuối, rồi mục 1.8.

Bước	Gõ	Thấy gì là đúng	Thấy khác thì
1. Dựng lab	Mục 1.6	`make status`: mọi pod `Running` hoặc `Completed`	Mục 1.8
2. Chạy thử 1 lượt ngắn	`scripts/k6.sh kb1-tu-hoc.js -e RATE=4 -e DURATION=1m`	Cuối bảng tóm tắt: `checks 100.00%`, dòng `{ kind:read }` có `p(95)` dưới 300 ms, `✓ http_req_failed 0.00%`	Có `✗` ngay đầu: API đang thêm bớt bản, chờ 1 phút chạy lại
3. Đo thăm dò	`scripts/k6.sh tang-dan.js -e MAX_RATE=40 -e STEP=45s`, mở http://grafana.hanni.localhost:18080/d/hanni-scaling	k6 tăng dần tốc độ, tự dừng khi p95 hoặc lỗi vượt ngưỡng; Grafana: số bản và CPU tăng	
4. Kiểm thử 2	`scripts/chay-kt2.sh k6/ket-qua/kt2 "10 24 36" 3` (khoảng 1 giờ, để máy yên)	Thư mục `k6/ket-qua/kt2/` có `dieu-kien.txt` và mỗi lần đo 3 tệp `muc-10-lan-1.txt`, `.json`, `-tai-nguyen.csv`...	Tệp `swap.csv` có số khác 0: máy thiếu RAM, kết quả lần đó không dùng được
5. Đọc 1 lần đo	`tail -40 k6/ket-qua/kt2/muc-36-lan-1.txt`	Đọc như mục 4.3: `p(95)`, `http_req_failed`, request/giây	
6. Kiểm thử 1	`scripts/kt1.sh k6/ket-qua/kt1.txt`, rồi `tail -1 k6/ket-qua/kt1.txt`	`Tổng: 30 tệp đúng, 0 tệp sai, 69174525 byte, 1.7 giây`	Có dòng `SAI`: học liệu bị đổi, xem tên tệp ở dòng đó
7. Máy yếu	Nhờ Đăng chạy trên GitHub Actions	Bạn đọc kết quả trong `ho-so-nop/data/raw/gha-09-11/` như bước 5	

6.2 Số liệu để nói khi bảo vệ

Câu cần nói	Số
Điểm gãy đo thăm dò	40 lượt/giây (khoảng 200 request/giây)
3 mức Kiểm thử 2	10, 24, 36 lượt/giây, mỗi mức 3 lần, mỗi lần đo 5 phút
Mức cao nhất, bản sau	p95 40 ± 2 ms, lỗi 0,06%, 179 request/giây

Câu cần nói	Số
Kiểm thử 1	3 lần đều 30/30 tệp khớp mã băm
Đăng nhập 40 lượt/giây	p95 72 ms, 0 lỗi
Quá tải 400 request/giây, trước khi sửa	430 request chờ kết nối CSDL, p95 5,0 giây
Máy đo	GitHub Actions Ubuntu 24.04.5 LTS, AMD EPYC 9V74 80-Core Processor, 4 luồng, 15983 MB RAM

Phần 7. Câu hỏi vấn đáp

Trả lời ngắn, có số, rồi mở tệp ghi ở dòng **Mở minh chứng** dưới mỗi câu. Thầy hỏi khác đi một chút thì vẫn dựa vào ý chính của câu trả lời.

Câu hỏi về phần của bạn

Vì sao dùng p95 mà không dùng trung bình?

Trung bình bị số đông nhanh che mất. p95 cho biết 5% người chậm nhất phải chờ bao lâu, đó mới là trải nghiệm tệ cần sửa.

Mở minh chứng: data/kt2-tong-hop.xlsx

Vì sao mỗi mức đo 3 lần?

Để thấy độ dao động. Ví dụ mức 36 lượt/giây bản trước có p95 57 ± 16 ms, lệch nhiều; bản sau 40 ± 2 ms, ổn định.

Mở minh chứng: Báo cáo nhóm mục 5.3

Chọn 3 mức tải thế nào?

Đo thăm dò tăng dần 10 → 80 lượt/giây, mức cao nhất còn đạt là 40. Lấy 25%, 60%, 90% của mức đó: 10, 24, 36 lượt/giây.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt2/tham-do/chon-muc.txt

Tạo tải kiểu "mở" khác kiểu "đóng" thế nào?

Mở gửi đều số lượt mỗi giây dù máy chủ chậm; đóng có số người ảo cố định, chờ trả lời mới gửi tiếp, nên khi máy chủ chậm tải tự giảm và che mất vấn đề.

Mở minh chứng: Báo cáo nhóm mục 5.7

Vì sao k6 chạy cùng máy là một giới hạn?

k6 cũng tốn CPU, tranh tài nguyên với hệ thống, nên điểm gãy thật có thể cao hơn. Nhóm ghi rõ giới hạn này; muốn chính xác hơn thì phát tải từ máy khác.

Mở minh chứng: Báo cáo nhóm mục 6.3

Thao tác có thể bị yêu cầu

Chạy 1 lượt k6 ngắn với RATE khác (scripts/k6.sh kb1-tu-hoc.js -e RATE=8 -e DURATION=1m), đọc p95 ở cuối kết quả; mở 1 thư mục dữ liệu gốc và chỉ số tương ứng trong bảng tổng hợp.

Mở minh chứng: cau-hinh-ma-nguon/hanni-server/infra/README.md, mục "Sửa nhanh khi bảo vệ"

Câu hỏi chung ai cũng phải trả lời được

Vì sao thử 20 kết nối không chứng minh phục vụ được 1.000 sinh viên?

Thời gian trả lời không tăng đều: từ 10 tới 40 lượt/giây p95 chỉ từ 23 lên 52 ms, thêm 10 lượt nữa đã 261 ms. Mức nhỏ chưa chạm giới hạn nào nên không nói gì về điểm gãy. Kết quả chỉ đúng cho đúng máy, dữ liệu, cấu hình đã đo.

Mở minh chứng: Báo cáo nhóm mục 5.3 (đo thăm dò), 6.2

Tài khoản khác người dùng cùng lúc thế nào?

Tài khoản là số người có thể dùng; người dùng cùng lúc mới tạo tải. 1.000 tài khoản sinh ra 63 người khi tự học, 300 khi cả lớp xem video, 600 khi thi thử đồng loạt.

Mở minh chứng: Báo cáo nhóm mục 2.1–2.2; so-do-va-bang/bang-tai-nguyen-ip-quyen.xlsx

Vì sao video không đi qua API?

Cả lớp xem video cần khoảng 888 Mbps trong khi API chỉ khoảng 56 request/giây. API chỉ kiểm quyền và ký liên kết, tệp do Nginx (thực tế là CDN) phát.

Mở minh chứng: Báo cáo nhóm mục 3.2; cau-hinh-ma-nguon/hanni-server/infra/k8s/app/media.yaml

Thêm máy chủ có hết quá tải không?

Không, nếu điểm nghẽn ở CSDL hoặc băng thông. Phải đo để biết điểm nghẽn trước: ở lab, nghẽn đầu tiên là hàng chờ kết nối CSDL, sửa xong mới tới CPU của API.

Mở minh chứng: Báo cáo nhóm mục 6.1; bảng Grafana "Mở rộng"

Bản sao lưu thế nào mới có giá trị?

Khi đã khôi phục thử, so số dòng và mã băm, đo được thời gian. Nhóm đã xoá hẳn CSDL rồi khôi phục: số dòng khớp, RTO 52 giây.

Mở minh chứng: Báo cáo nhóm mục 5.4; data/rpo-rto.xlsx

Lên 10.000 hay 100.000 sinh viên thì làm gì?

10.000: CSDL máy riêng có bản sao, kho tệp + CDN bắt buộc, khoảng 5 bản API. 100.000: nhiều vùng, tự chuyển CSDL khi hỏng, tách kho phân tích. Chuyển mức khi số đo chạm ngưỡng, không làm trước.

Mở minh chứng: Báo cáo nhóm mục 6.4

Phần 8. Việc cần làm trước khi nộp và bảo vệ

Trước khi nộp

Báo cáo cá nhân của bạn `ho-so-nop/bao-cao/ca-nhan/RIPT1304-20261-01_NhomXX_06_B24DCCC007_BaoCaoCaNhan.docx` **đã điền đủ** theo mẫu: mã sinh viên, lớp, ngày, họ tên ở mục 10. Chỉ còn số nhóm (chỗ `XX` trong tên tệp và dòng "Nhóm"), Đăng điền khi thầy công bố rồi nộp cả hồ sơ lên Drive của thầy.

Việc của bạn: đọc báo cáo nhóm `ho-so-nop/bao-cao/Bao-cao-nhom.pdf` và báo cáo cá nhân của bạn để nắm mình đã kê khai những gì.

Trình bày 3 phút

Lúc bảo vệ, mỗi người có vài phút trình bày phần mình. Khung dưới đây đi đúng thứ tự thầy chấm: việc gì, bằng chứng ở đâu, số đo bao nhiêu, giới hạn là gì. Tập nói to 2–3 lần, bấm giờ.

Thời gian	Nói gì	Mở gì
0:00–0:30	k6 giả lập sinh viên gửi request đều đặn (mô hình mở), đo thời gian trả lời; tiêu chí đạt: 95% lượt đọc dưới 300 ms, lỗi dưới 0,5%	infra/k6/lib.js
0:30–1:10	Đo thăm dò tìm điểm gãy khoảng 40 lượt/giây, chọn 3 mức 10, 24, 36 lượt/giây (khoảng 25%, 60%, 90% điểm gãy)	ket-qua-kt2/tham-do/
1:10–2:10	Kiểm thử 2: mỗi mức 3 lần × 5 phút, 6 bản API, làm nóng 1 phút, kiểm tra không swap; mức cao nhất p95 40 ± 2 ms, lỗi 0,06%	data/kt2-tong-hop.xlsx
2:10–2:40	Kiểm thử 1: tải 30 tệp như máy khách, so mã băm, 3 lần đều 30/30; bảng kiểm thử 62 dòng có giờ chạy	bang-kiem-thu.xlsx
2:40–3:00	Giới hạn: k6 chạy chung máy với hệ thống nên số chỉ đúng cho lab; lỗi đo đầu tiên là do đo lúc đang thêm bản, đã sửa bằng chờ đủ bản	Mục 4.6

Trước buổi bảo vệ

1. Tập **mở đúng minh chứng trong 10 giây**: biết tệp kết quả của mình nằm ở đâu trong `ho-so-nop/` (bảng "Sản phẩm của bạn" ở phần 2).
2. Tập **1–2 thao tác** ở phần "Sửa và cấu hình trực tiếp": nói được tệp nào, dòng nào, lệnh áp dụng, cách kiểm tra.
3. Tập trả lời các câu ở phần Câu hỏi vấn đáp bằng lời của mình, có số.
4. Xem lại phần 1 (kiến thức nền): thầy có thể hỏi kiến trúc chung, không chỉ phần của bạn.

> Câu chốt khi bị hỏi khó: "Theo số đo của nhóm trên lab, trong điều kiện … thì … ; giới hạn là … ; muốn chắc hơn thì đo thêm …". Trả lời theo số đo và nêu giới hạn luôn được đánh giá cao hơn khẳng định chung chung.